



US 20250278091A1

(19) **United States**

(12) **Patent Application Publication**
Sajjan et al.

(10) **Pub. No.: US 2025/0278091 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **OBSTACLE TO PATH ASSIGNMENT FOR
AUTONOMOUS SYSTEMS AND
APPLICATIONS**

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA
(US)

(72) Inventors: **Neeraj Sajjan**, Sunnyvale, CA (US);
Mehmet K. Kocamaz, San Jose, CA
(US); **Junghyun Kwon**, Santa Clara,
CA (US); **Sangmin Oh**, San Jose, CA
(US); **Minwoo Park**, Saratoga, CA
(US); **David Nister**, Bellevue, WA (US)

(21) Appl. No.: **19/191,529**

(22) Filed: **Apr. 28, 2025**

Related U.S. Application Data

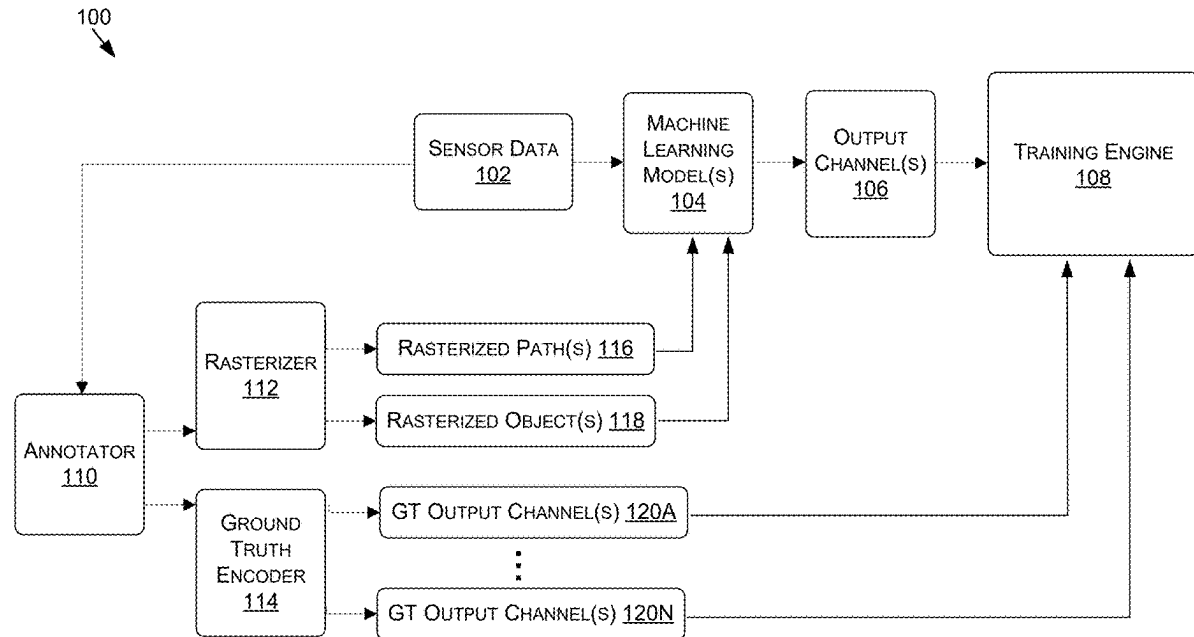
(63) Continuation of application No. 17/565,837, filed on
Dec. 30, 2021, now Pat. No. 12,346,119.

Publication Classification

(51) **Int. Cl.**
G05D 1/00 (2024.01)
G06N 3/08 (2023.01)
(52) **U.S. Cl.**
CPC **G05D 1/622** (2024.01); **G06N 3/08**
(2013.01); **G05D 2101/15** (2024.01); **G05D**
2101/20 (2024.01)

(57) **ABSTRACT**

In various examples, one or more output channels of a deep neural network (DNN) may be used to determine assignments of obstacles to paths. To increase the accuracy of the DNN, the input to the DNN may include an input image, one or more representations of path locations, and/or one or more representations of obstacle locations. The system may thus repurpose previously computed information—e.g., obstacle locations, path locations, etc.—from other operations of the system, and use them to generate more detailed inputs for the DNN to increase accuracy of the obstacle to path assignments. Once the output channels are computed using the DNN, computed bounding shapes for the objects may be compared to the outputs to determine the path assignments for each object.



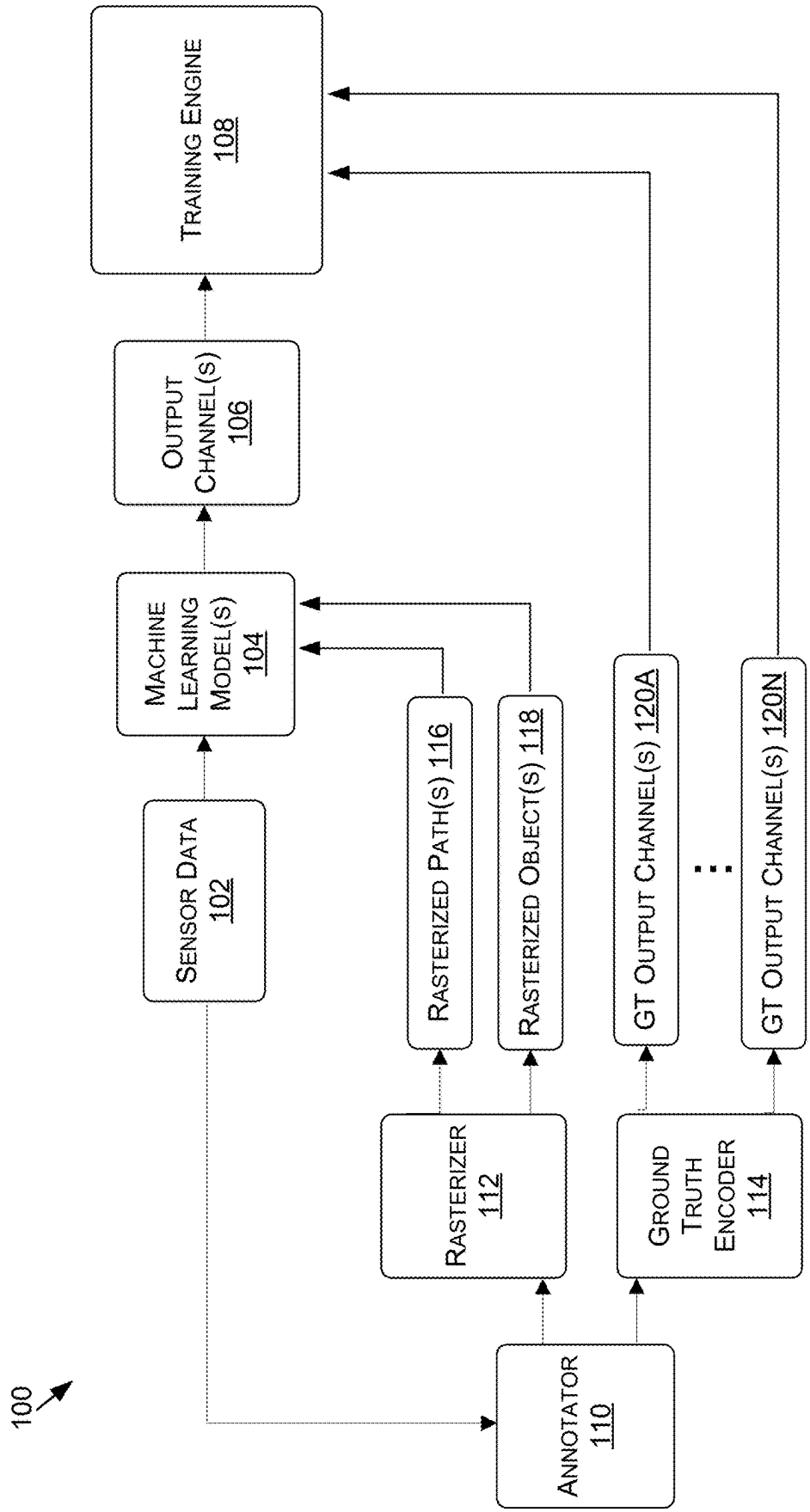


FIGURE 1

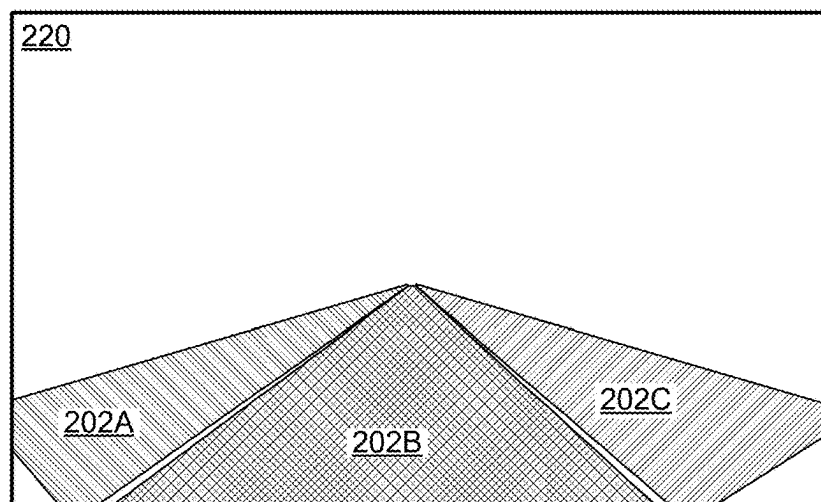
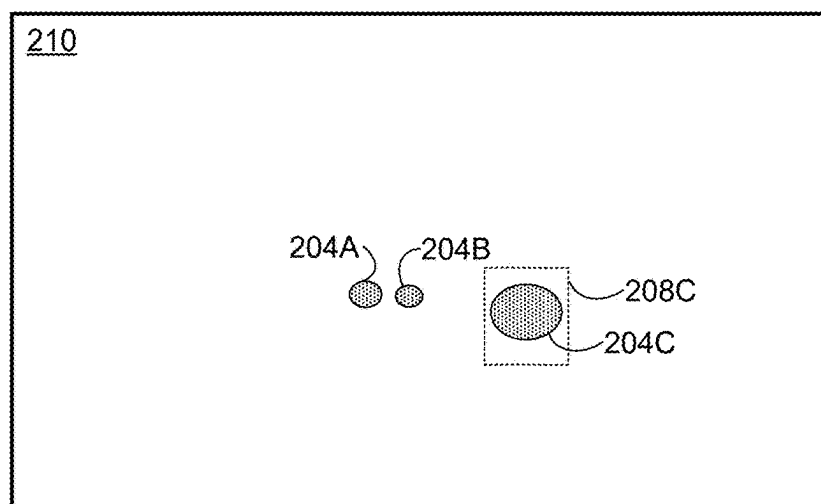
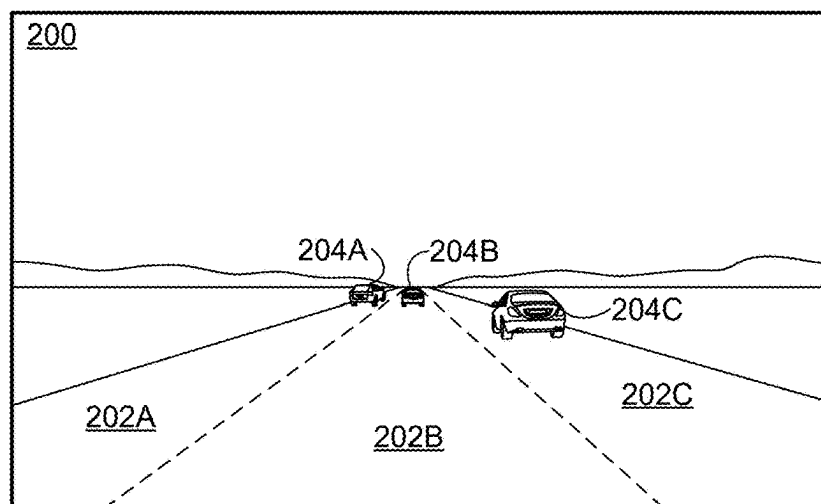


FIGURE 2

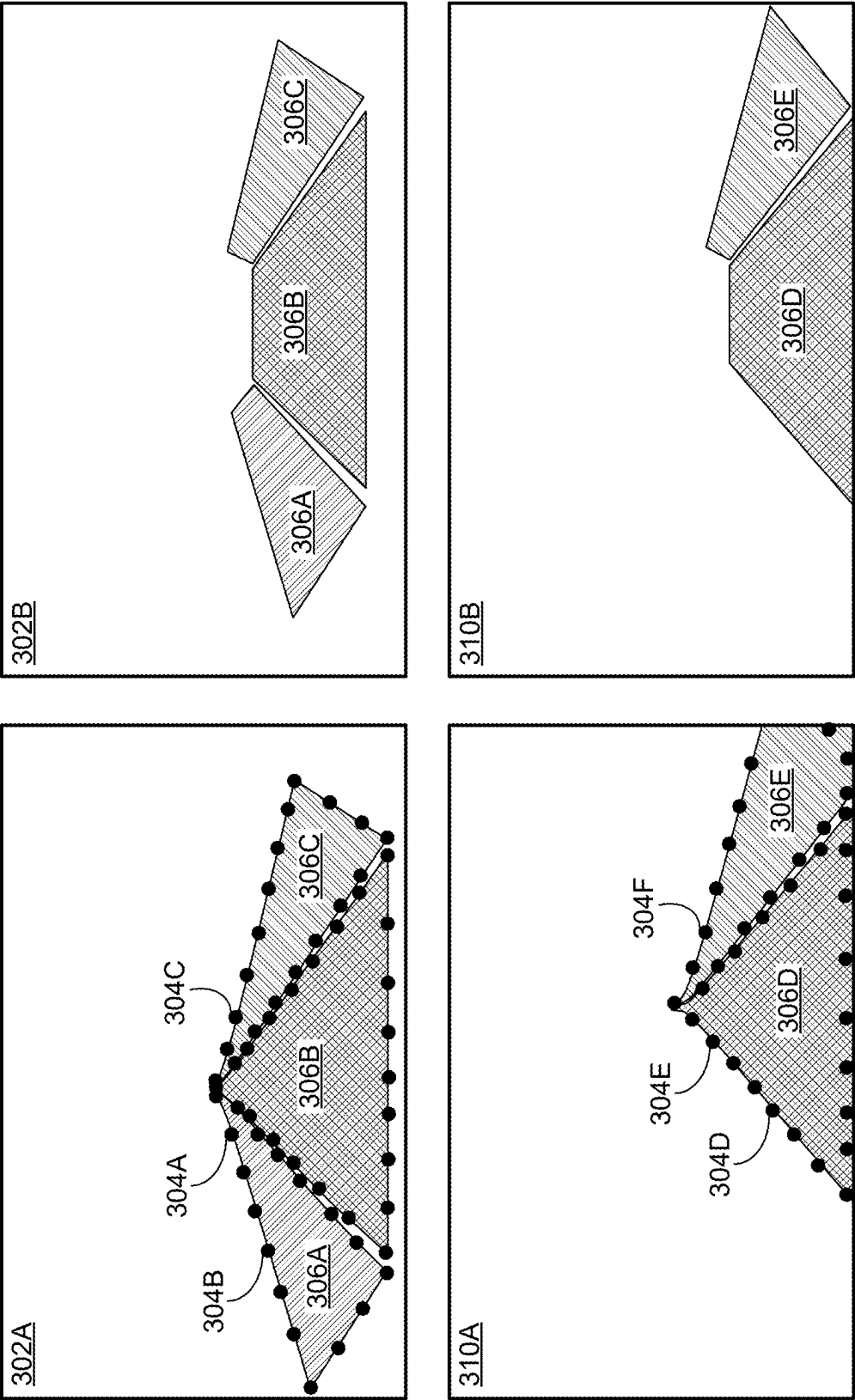
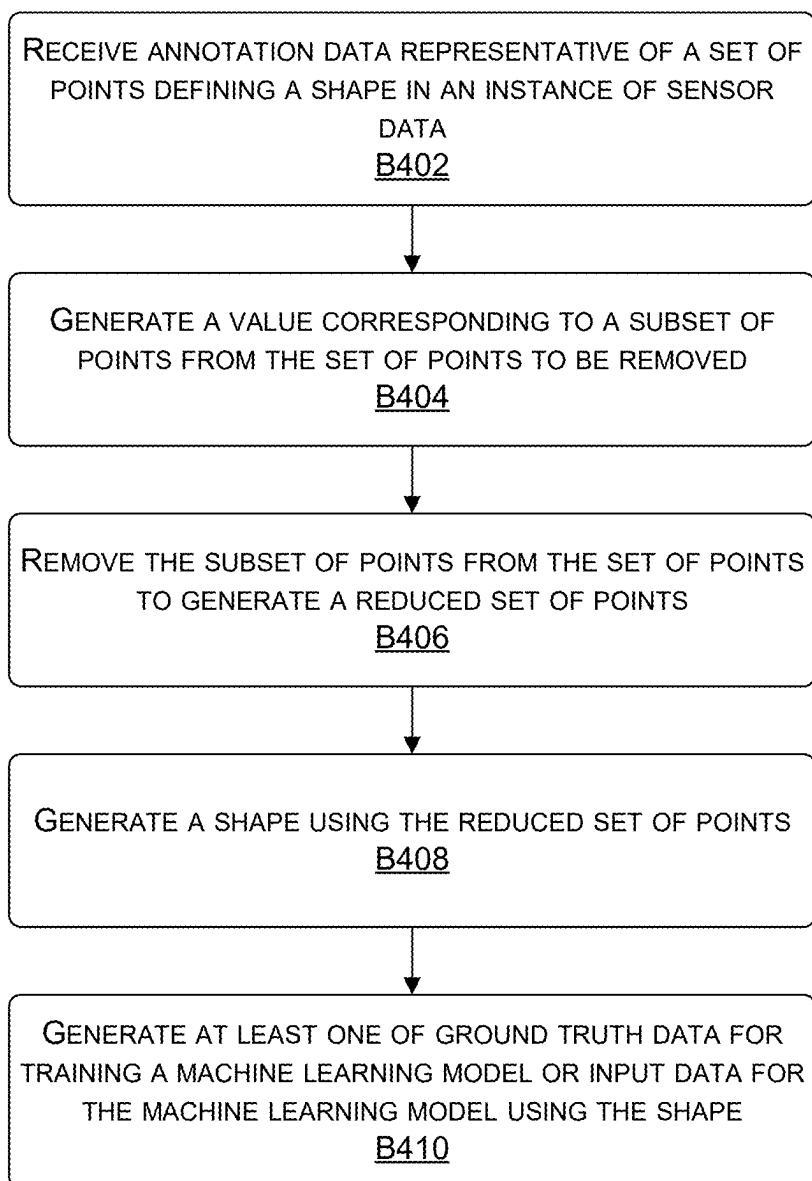


FIGURE 3

400
↓**FIGURE 4**

500
↓

RECEIVE ANNOTATION DATA INDICATIVE OF ONE OR MORE BOUNDING SHAPE LOCATIONS CORRESPONDING TO ONE OR MORE OBJECTS AND ONE OR MORE ANNOTATIONS CORRESPONDING TO PATH LOCATIONS OF ONE OR MORE PATHS

B502

GENERATE, BASED AT LEAST IN PART ON THE ANNOTATION DATA, ONE OR MORE FIRST IMAGES INDICATIVE OF OBJECT LOCATIONS AND ONE OR MORE SECOND IMAGES INDICATIVE OF PATH LOCATIONS

B504

GENERATE, BASED AT LEAST IN PART ON THE ANNOTATION DATA, GROUND TRUTH OUTPUT DATA INDICATIVE OF ONE OR MORE OBJECT LOCATIONS WITHIN THE ONE OR MORE PATHS

B506

DURING TRAINING OF A MACHINE LEARNING MODEL, USE THE ONE OR MORE FIRST IMAGES AND THE ONE OR MORE SECOND IMAGES AS INPUTS TO THE MACHINE LEARNING MODEL AND THE GROUND TRUTH OUTPUT DATA AS GROUND TRUTH

B508

FIGURE 5

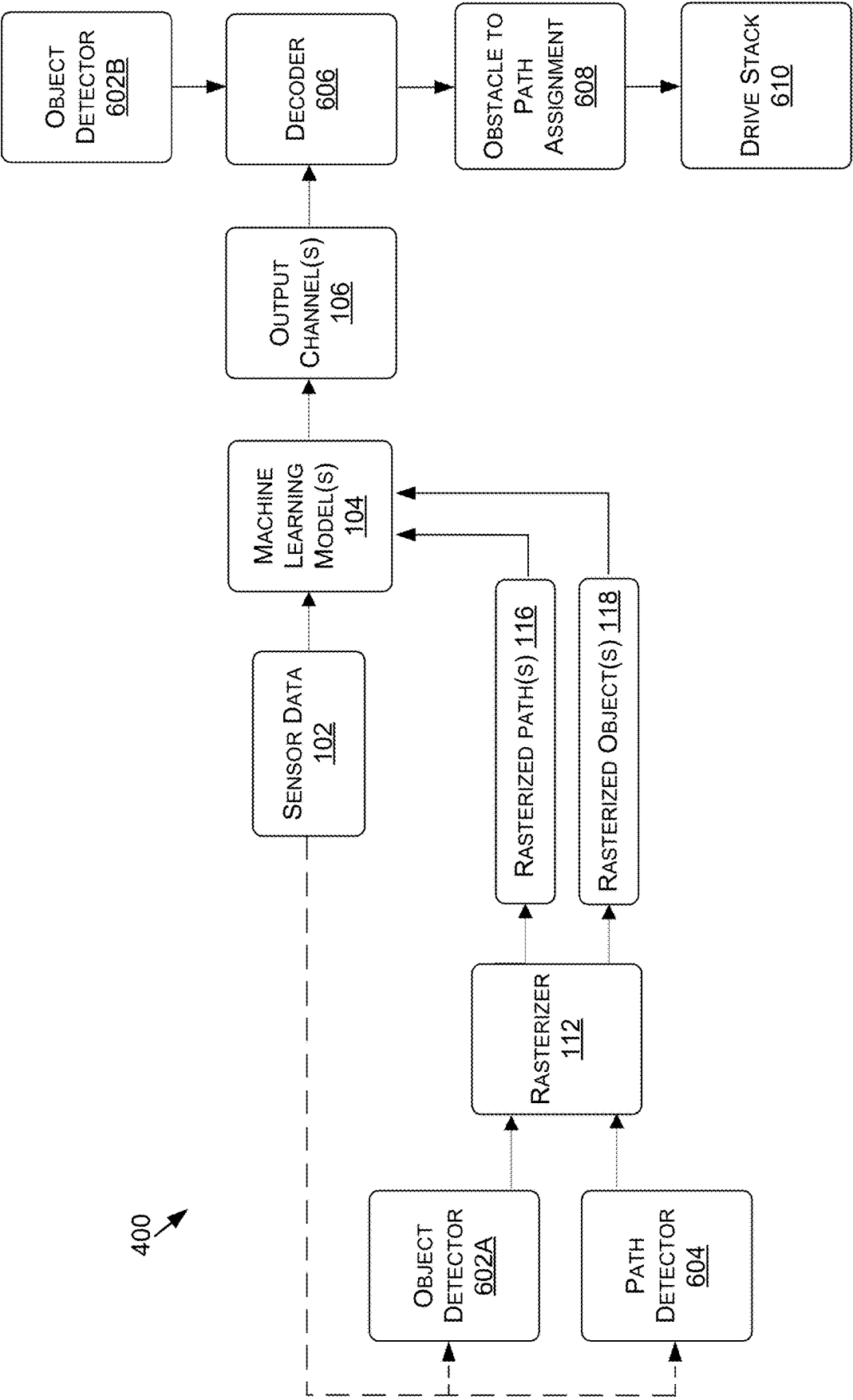


FIGURE 6

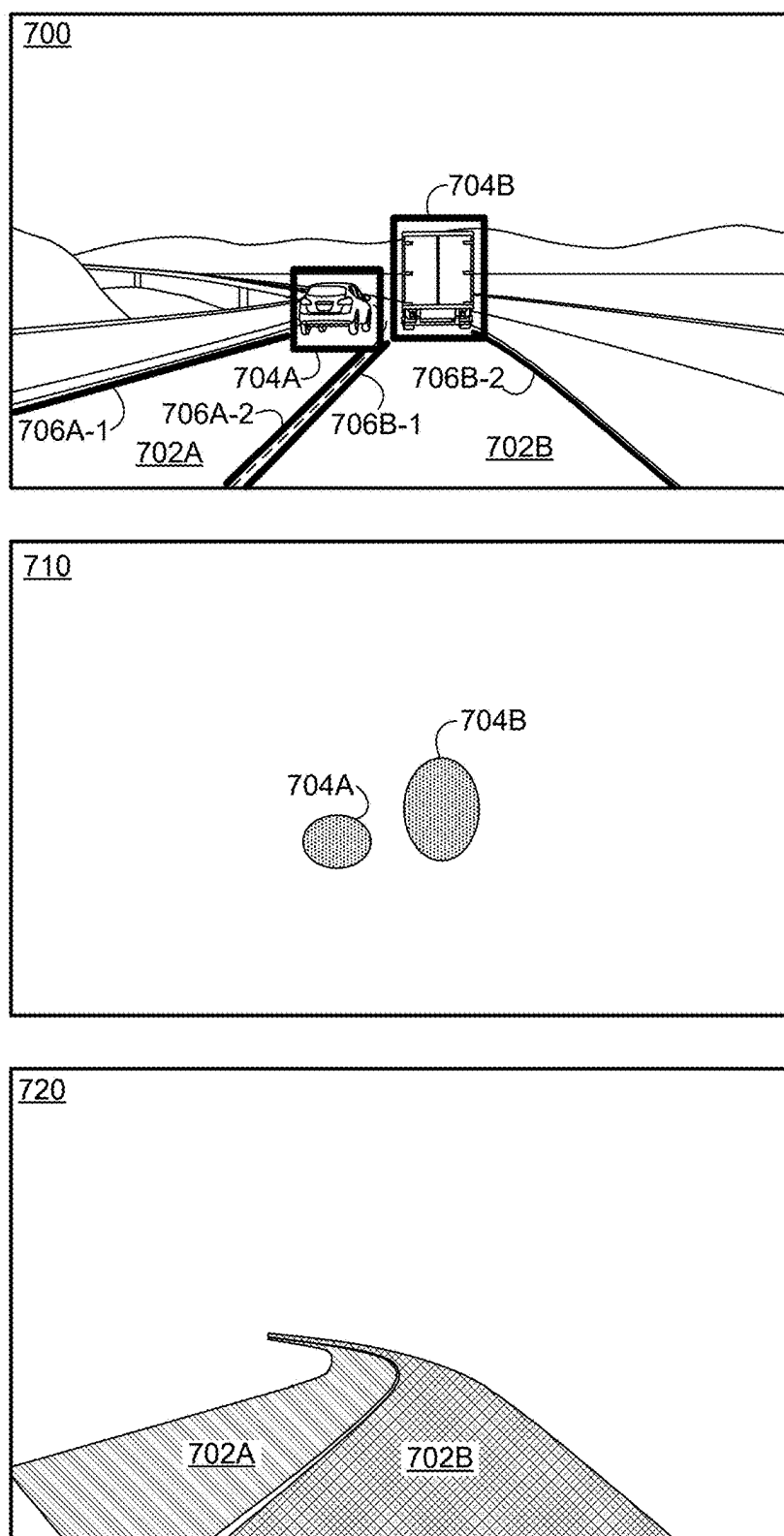


FIGURE 7

800
↓

GENERATE, FROM A SENSOR DATA REPRESENTATION, A FIRST INPUT REPRESENTATIVE OF ONE OR MORE PATH LOCATIONS OF ONE OR MORE PATHS IN THE SENSOR DATA REPRESENTATION AND A SECOND INPUT REPRESENTATIVE OF OBJECT LOCATIONS IN THE SENSOR DATA REPRESENTATION

B802

↓

COMPUTE, USING A NEURAL NETWORK AND BASED AT LEAST IN PART ON THE SENSOR DATA REPRESENTATION, THE FIRST INPUT, AND THE SECOND INPUT, ONE OR MORE OUTPUTS INDICATIVE OF THE OBJECT LOCATIONS WITHIN THE ONE OR MORE PATHS

B804

↓

FOR AT LEAST ONE DETECTED OBJECT, COMPARE A BOUNDING SHAPE CORRESPONDING TO THE AT LEAST ONE DETECTED OBJECT TO AT LEAST ONE OUTPUT OF THE ONE OR MORE OUTPUTS

B806

↓

BASED AT LEAST IN PART ON THE COMPARISON, DETERMINE A PATH OF THE ONE OR MORE PATHS THAT THE AT LEAST ONE DETECTED OBJECT IS LOCATED

B808

↓

PERFORM ONE OR MORE CONTROL OPERATIONS FOR A MACHINE BASED AT LEAST IN PART ON THE AT LEAST ONE DETECTED OBJECT BEING LOCATED IN THE PATH

B810**FIGURE 8**

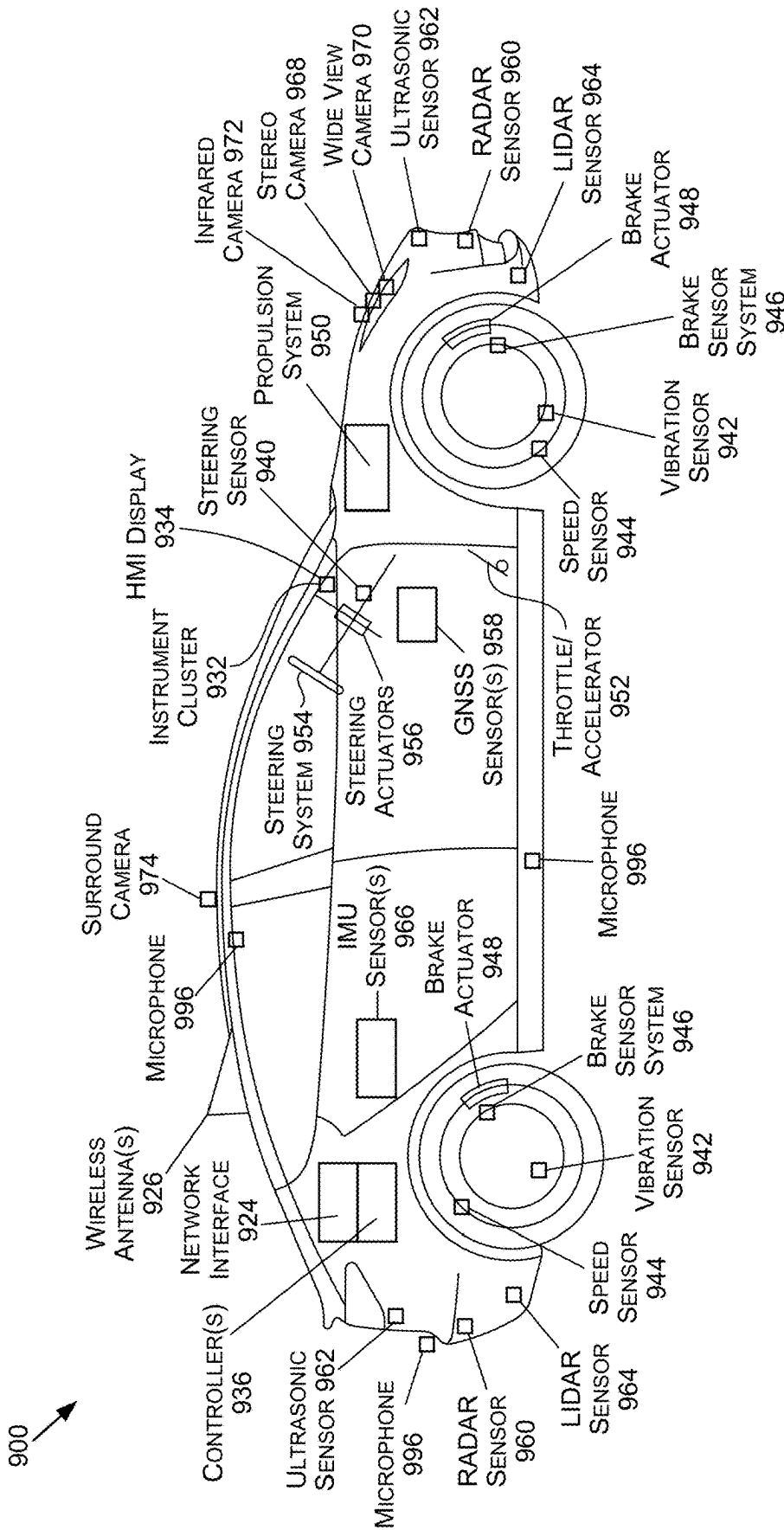


FIGURE 9A

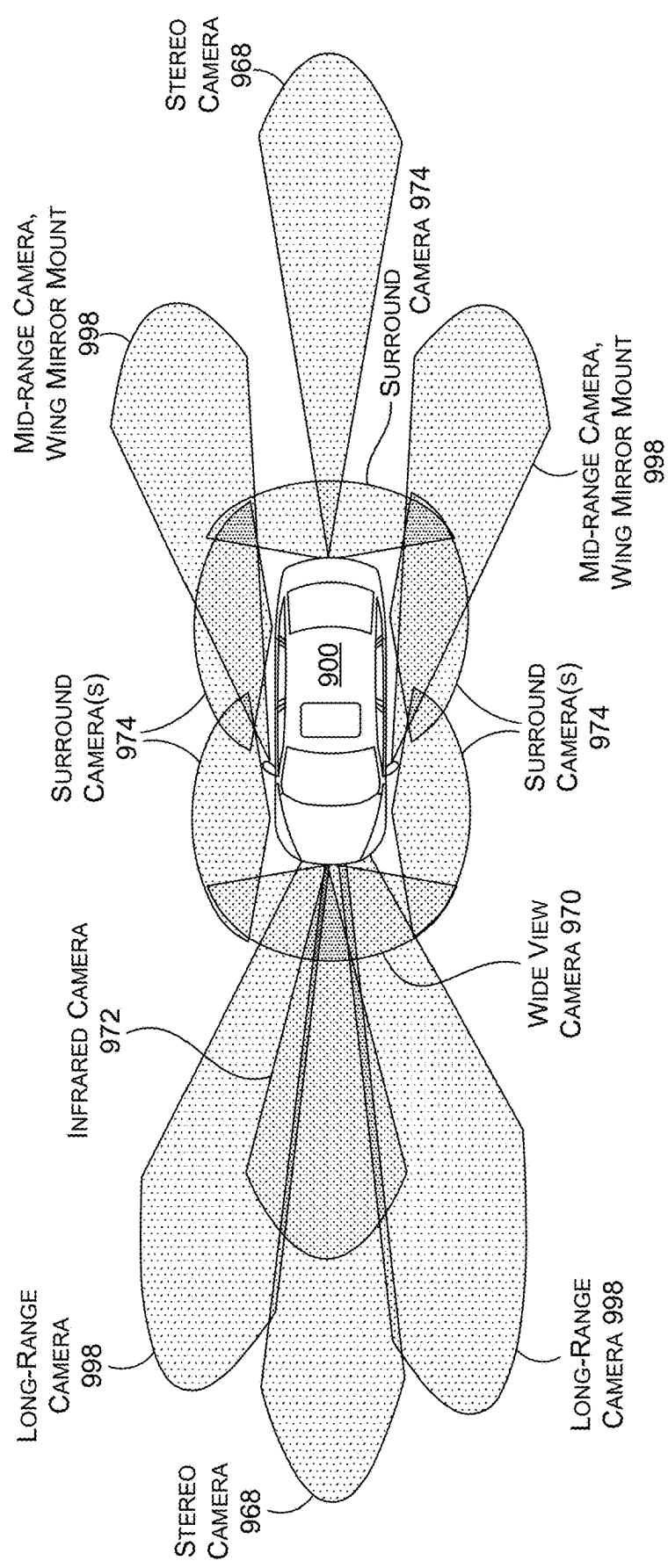


FIGURE 9B

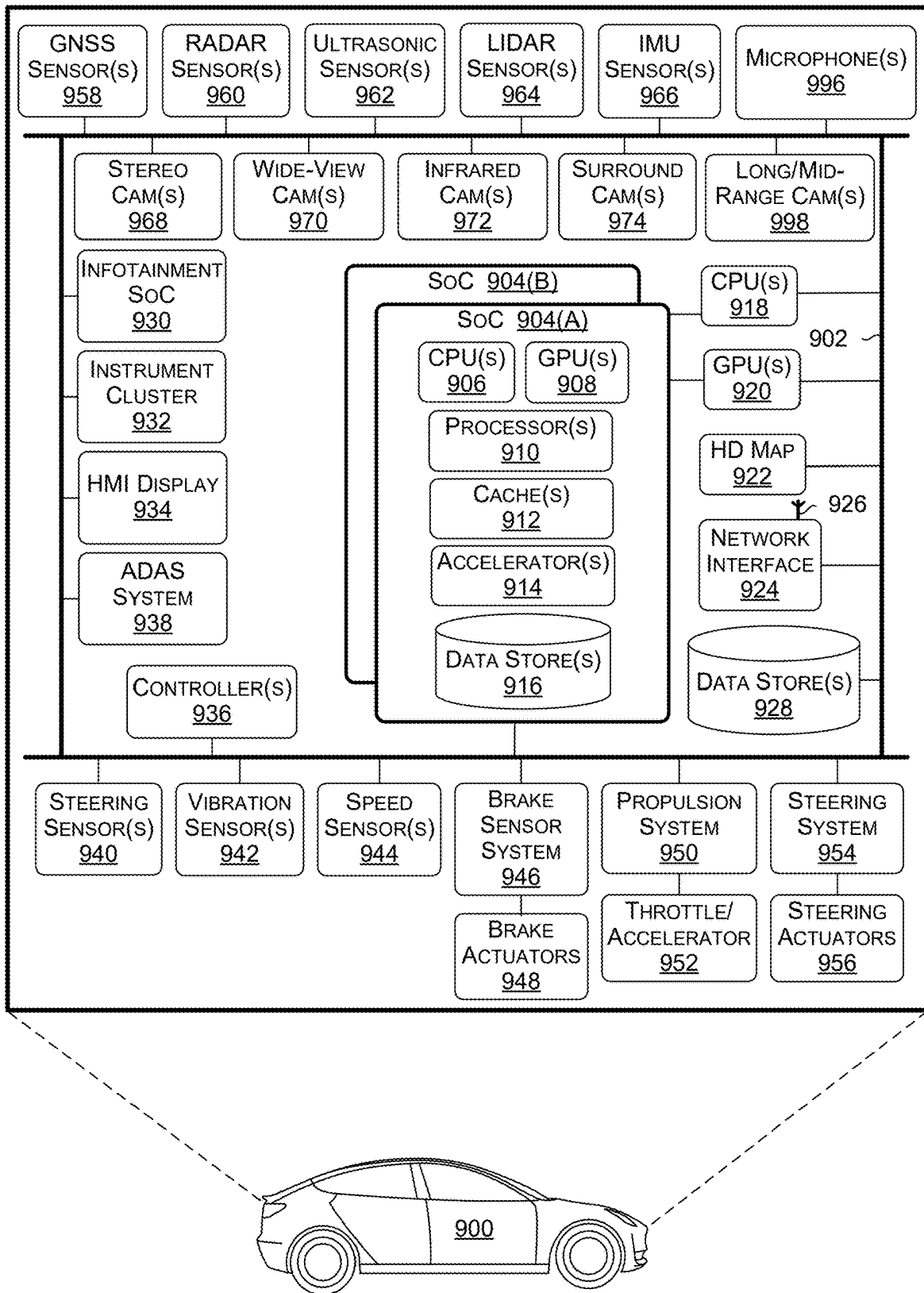


FIGURE 9C

976 ↗

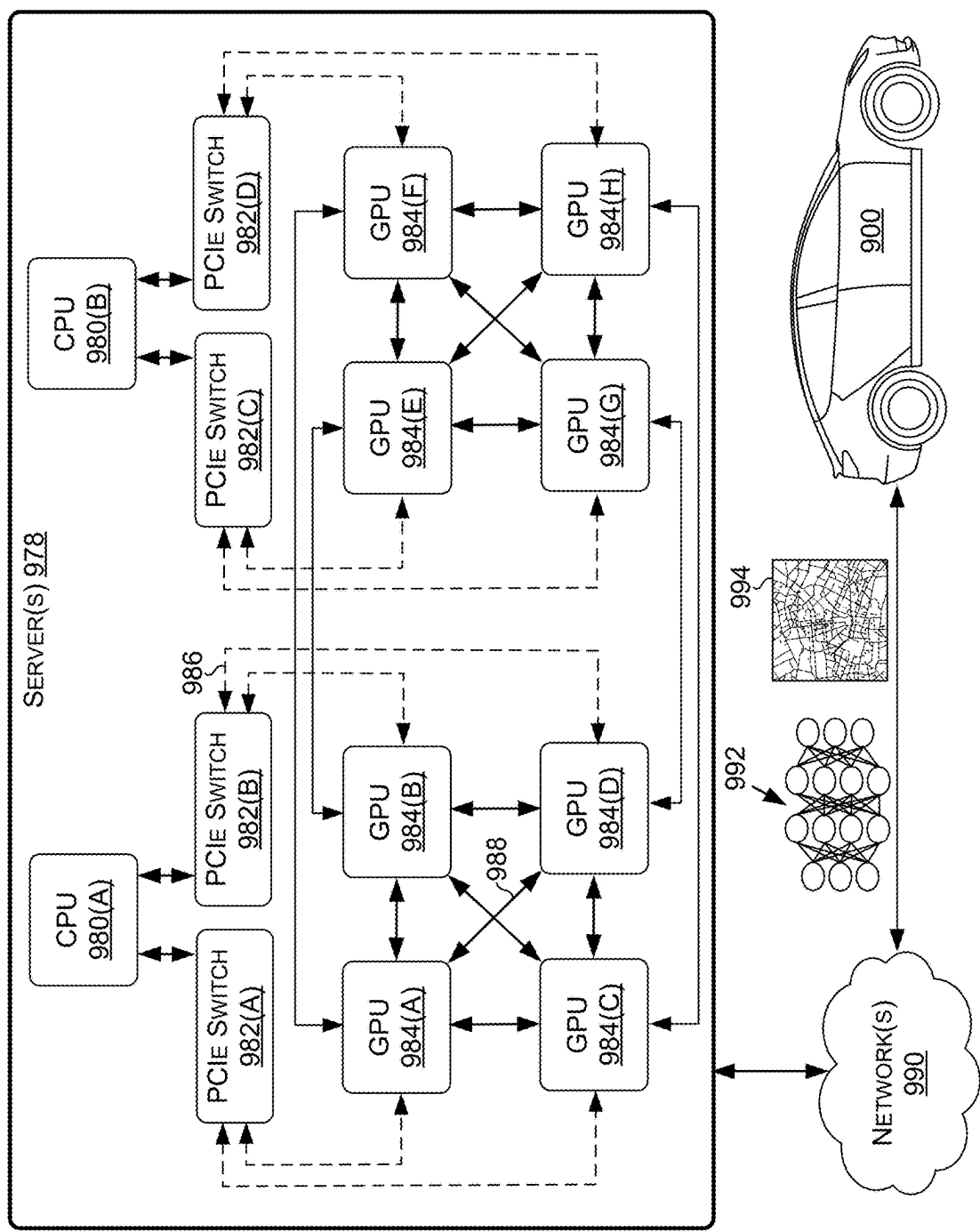


FIGURE 9D

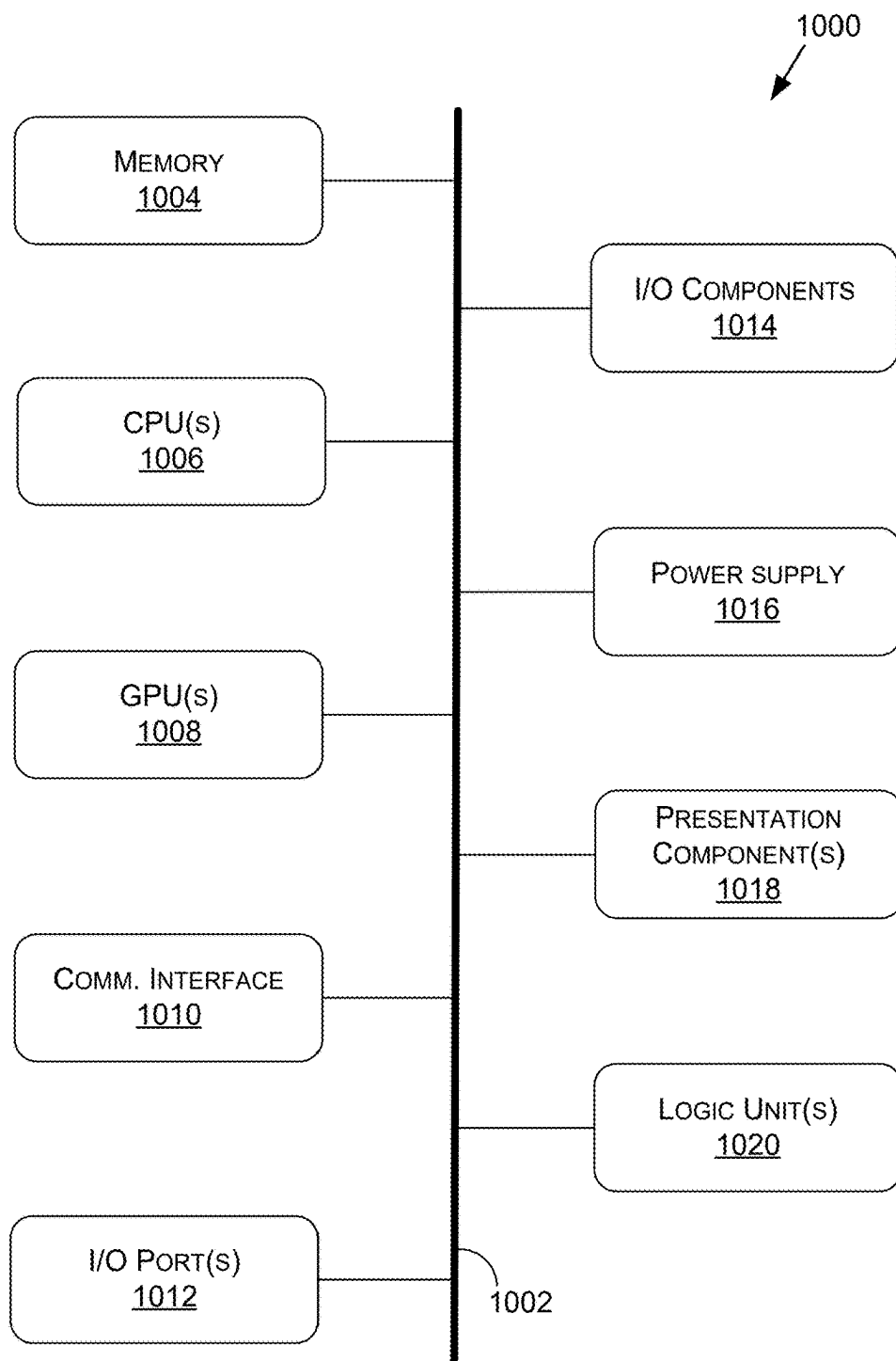


FIGURE 10

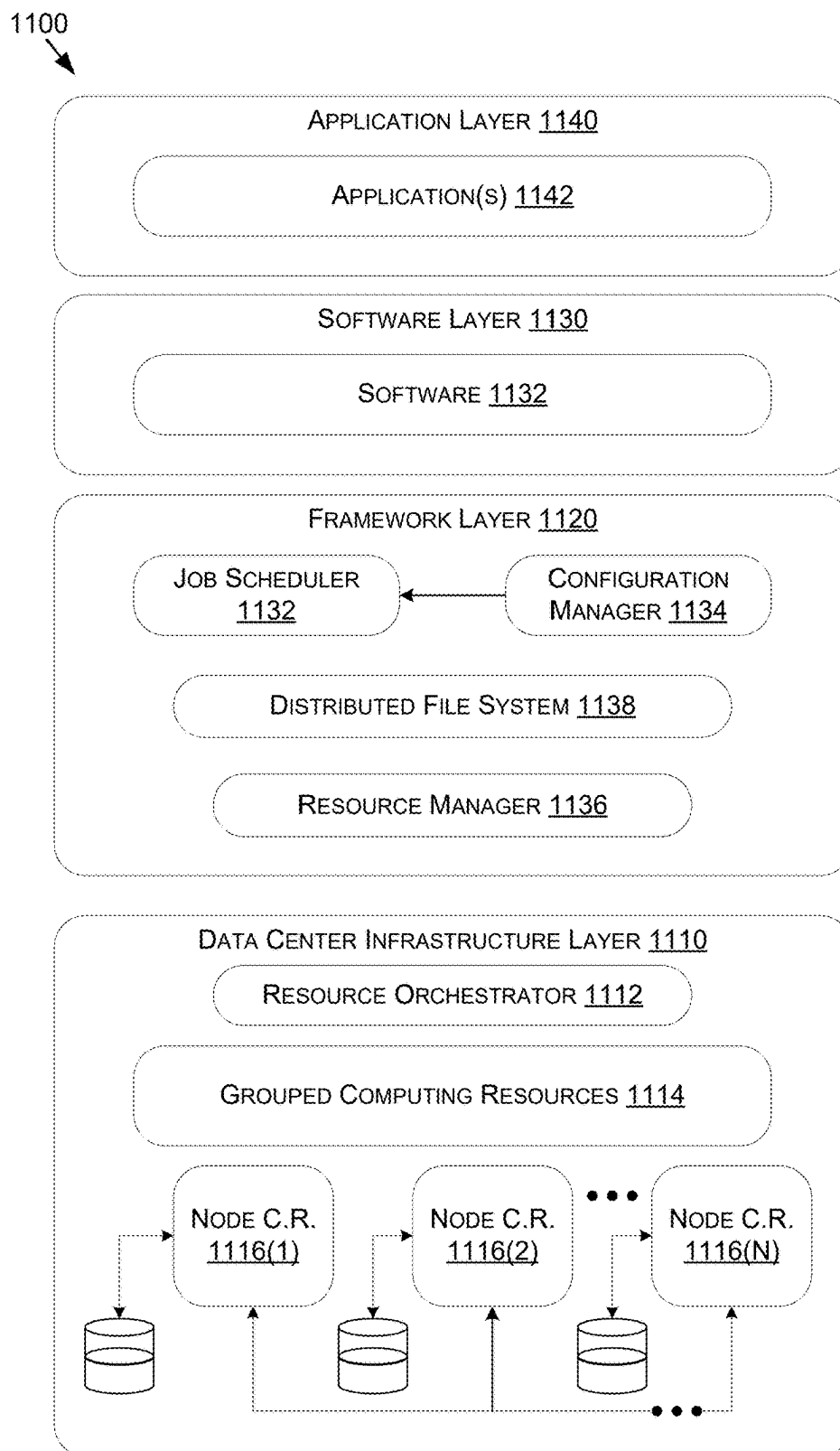


FIGURE 11

OBSTACLE TO PATH ASSIGNMENT FOR AUTONOMOUS SYSTEMS AND APPLICATIONS

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is a continuation of U.S. patent application Ser. No. 17/565,837, filed Dec. 30, 2021, which is hereby incorporated by reference in its entirety.

BACKGROUND

[0002] Autonomous and semi-autonomous driving systems, as well as advanced driver assistance systems (ADAS) use sensors, such as cameras, LiDAR, RADAR, etc., to perform various tasks—such as lane keeping, lane changing, lane assignment, camera calibration, and localization. For example, for these systems to operate independently and efficiently, an understanding of the surrounding environment of the vehicle may be generated. This understanding may include information as to locations of objects and/or obstacles in the environment with respect to various demarcations—such as lanes, road boundaries, intersections, and/or the like. Once an obstacle to path assignment is determined, this information may be used by a machine (e.g., a vehicle, robot, etc.) when making decisions, such as what path or trajectory to follow, whether to speed up or slow down, whether a lane change or other maneuver is currently safe or necessary, and/or the like.

[0003] Some conventional methods and systems perform the detection of lanes as a separate task from the detection of objects, and then rely on post-processing to determine object to lane assignments. These methods generally require the preliminary operations of detecting objects in two-dimensional (2D) space and (separately) identifying lanes in three-dimensional (3D) space, prior to determining an intersection of the objects and lanes. When determining the intersections, the 3D locations of the lanes are projected to 2D space and compared with the 2D bounding shapes. Where a bounding shape overlaps a lane, the obstacle associated with the bounding shape is determined to be located in the lane. However, this post-processing of the identified lanes and objects requires significant use of computing resources and additional processing time which presents limitations to a real-time or near real-time autonomous or semi-autonomous driving system. For example, the end-to-end sequence of processing sensor data to determine object locations, then processing sensor data to determine path locations, and then synchronizing and aligning the two detection types requires increased processing, compute, and energy consumption. In addition, the outputs may be noisy and inaccurate, as the projection from 3D to 2D may result in inaccurate triangulation—e.g., because the projection is performed with a flat world assumption, where the path includes curves, dips, hills, and/pitch, the resulting projection may be inaccurate. These issues may be compounded where the path has high curvature, the objects are far away, the objects are small and/or appear small, etc.

[0004] In other conventional systems, a deep neural network (DNN) may be used to compute a multi-channel output that may be used for semantic classification of objects to paths. For example, such systems may compute an output per lane, and assign an object (e.g., a bounding shape) to the lane with the highest activation. Using this approach, the

DNN may be trained to handle different scenarios that are less common and/or are complex. However, performing obstacle to path assignment solely from an input image forces the DNN to intrinsically interpret lane and object information from an input image, as well as determine associations between them. This may increase the compute requirements and complexity of the DNN, as well as lead to less accurate obstacle to path assignments in scenarios where objects are far away and/or on paths with high curvature. In such cases, the objects would occupy very few pixels making it challenging for the DNN to glean spatial location and size of each object and/or lane with high degrees of accuracy or precision. Attempts to solve this may include training the DNN with a large amount of data so that the DNN may learn from many scenarios, but this also requires a larger (e.g., more layers) DNN and extensive testing to find a properly designed loss function to allow for accurate DNN performance. Increasing the size and complexity of the DNN may result in increased inference—and thus increased compute and process time—which may make the DNN less capable of operating in real-time or near real-time. In addition, generating enough data to capture all these challenging scenarios requires a large amount of annotated data which is expensive and time consuming.

SUMMARY

[0005] Embodiments of the present disclosure relate to obstacle to path assignment for autonomous machine systems and applications. Systems and methods are disclosed that use one or more output channels of a deep neural network (DNN) to determine assignments of obstacles (e.g., vehicles, bicyclists, pedestrians, etc.) to paths (e.g., lanes on a road, pathways, walkways in indoor spaces, etc.). For example, to increase the accuracy of the DNN, the input to the DNN may include an input image, one or more representations of path locations, and/or one or more representations of obstacle locations. As a result, the DNN may more accurately compute outputs indicative of obstacles to path assignments as the DNN processes more explicit representations of the path and/or obstacle locations—as opposed to traditional systems that use the image only as input to compute outputs indicative of obstacle to path assignments. In this way, the system may repurpose previously computed information—e.g., obstacle locations, path locations, etc.—from other operations of the system, and use them to generate more detailed inputs for the DNN to increase accuracy of the obstacle to path assignments. In some embodiments, the path representations may include one or more rasterized images—e.g., one image channel per lane—that indicate lane shape and/or semantic information (e.g., ego-lane, left of ego-lane, right of ego-lane, etc.), and the object representations may include one or more rasterized images that indicate object locations and shape (e.g., using bounding shapes). In some embodiments, the rasterized object images may include rasterizing only a portion of a bounding shape for each object—e.g., an ellipse, circle, square, and/or other sub-regions of the bounding shape—in order to increase the accuracy of the predictions (e.g., because some portions of a bounding shape, such as corners, may include portions of an environment other than the object).

[0006] Once the output channels are computed using the DNN, computed bounding shapes (e.g., previously computed using another DNN or object detection algorithm, or

computed using the DNN) for the object(s) may be compared to the outputs to determine the assignments for each object. As a result, due to the increased accuracy of the outputs as a result of the additional input information for the DNN, the obstacle to path assignments may be more accurate and thus allow for safer and more accurate downstream operations of the autonomous or semi-autonomous system when relying on the obstacle to lane assignment information.

[0007] In some embodiments, to increase the size of the training data set to account for less common scenarios or path detection inaccuracies, data augmentation may be used. For example, lane boundaries may be demarcated using points, and these points may be used to generate polylines that may define the boundaries of lanes. When generating these annotations during training, the annotations may be accurate, may extend the full length of a path as visible in a given image, and may be shaped appropriately. In deployment, however, the path location predictions may be less accurate, may be abbreviated (e.g., to due to occlusion, a poor detection, etc.), and/or may be shaped incorrectly. To account for this, in embodiments, a random number generator may be used to randomly remove some percentage or amount of the annotated points to change the shape and/or size of the paths. For example, some percentage of the points from the top of the image (e.g., corresponding to a distance further from the sensor) may be removed, and the remaining points may be used to generate the polylines that define the path shapes. These polyline-defined shapes may then be used by a rasterizer to generate the rasterized images representing path locations that may be used as input to the DNN. As such, by augmenting the data, the training data set may be more representative of the data that will be generated in deployment (e.g., accounting for inaccuracies or other errors), thus allowing the DNN to perform accurately even in instances where the input data is substandard.

BRIEF DESCRIPTION OF THE DRAWINGS

[0008] The present systems and methods for obstacle to path assignment for autonomous machine applications are described in detail below with reference to the attached drawing figures, wherein:

[0009] FIG. 1 is an example data flow diagram illustrating an example process for training a neural network to compute outputs usable for assigning obstacles to paths, in accordance with some embodiments of the present disclosure;

[0010] FIG. 2 depicts example visualizations of inputs to a neural network, in accordance with some embodiments of the present disclosure;

[0011] FIG. 3 depicts example visualizations of augmenting training data annotations to simulate live perception, in accordance with some embodiments of the present disclosure;

[0012] FIG. 4 is a flow diagram illustrating a method for augmenting training data annotations to simulate live perception, in accordance with some embodiments of the present disclosure;

[0013] FIG. 5 is a flow diagram illustrating a method for training a neural network to compute outputs usable for assigning obstacles to paths, in accordance with some embodiments of the present disclosure;

[0014] FIG. 6 is an example data flow diagram illustrating an example process for using a neural network to aid in the

assignment of obstacles to paths, in accordance with some embodiments of the present disclosure;

[0015] FIG. 7 depicts example visualizations of inputs to a neural network, in accordance with some embodiments of the present disclosure;

[0016] FIG. 8 is a flow diagram illustrating a method of using a neural network to aid in the assignment of obstacles to paths, in accordance with some embodiments of the present disclosure;

[0017] FIG. 9A is an illustration of an example autonomous vehicle, in accordance with some embodiments of the present disclosure;

[0018] FIG. 9B is an example of camera locations and fields of view for the example autonomous vehicle of FIG. 9A, in accordance with some embodiments of the present disclosure;

[0019] FIG. 9C is a block diagram of an example system architecture for the example autonomous vehicle of FIG. 9A, in accordance with some embodiments of the present disclosure;

[0020] FIG. 9D is a system diagram for communication between cloud-based server(s) and the example autonomous vehicle of FIG. 9A, in accordance with some embodiments of the present disclosure;

[0021] FIG. 10 is a block diagram of an example computing device suitable for use in implementing some embodiments of the present disclosure; and

[0022] FIG. 11 is a block diagram of an example data center suitable for use in implementing some embodiments of the present disclosure.

DETAILED DESCRIPTION

[0023] Systems and methods are disclosed related to obstacle to path assignment for autonomous machine applications. Although the present disclosure may be described with respect to an example autonomous vehicle **900** (alternatively referred to herein as “vehicle **900**” or “ego-machine **900**,” an example of which is described with respect to FIGS. 9A-9D), this is not intended to be limiting. For example, the systems and methods described herein may be used by, without limitation, non-autonomous vehicles, semi-autonomous vehicles (e.g., in one or more advanced driver assistance systems (ADAS)), piloted and un-piloted robots or robotic platforms, warehouse vehicles, off-road vehicles, vehicles coupled to one or more trailers, flying vessels, boats, shuttles, emergency response vehicles, motorcycles, electric or motorized bicycles, aircraft, construction vehicles, underwater craft, drones, and/or other vehicle types. In addition, although the present disclosure may be described with respect to obstacle to path assignment for autonomous or semi-autonomous machines, this is not intended to be limiting, and the systems and methods described herein may be used in augmented reality, virtual reality, mixed reality, robotics, security and surveillance, autonomous or semi-autonomous machine applications, and/or any other technology spaces where assignment of detected objects or obstacles to paths or portions of an environment may be used.

[0024] Systems and methods of the present disclosure use detected paths and detected bounding shapes to rasterize binary maps or images that may be provided as input to a deep neural network (DNN) along with sensor data. In this way, the DNN may use the path and object information, as well as the sensor data, to fill in the gaps or remove

noise—thus resulting in increased performance, accuracy, and precision for obstacle to path assignment. The obstacle to path assignment may thus be formulated as a binary classification problem per path (e.g., lane), which may include, for example and without limitation, for every N detected lanes, the DNN may output N channels—e.g., one for each path. For each path, binary classification may be performed to determine whether each object or obstacle belongs to that specific path or not. As such, there is no competition between the paths which may ensure that objects or obstacles are assigned to whichever path has the highest activation for them.

[0025] Knowing that the detected paths and bounding shapes can be noisy and/or inaccurate (e.g., due to noise or inaccuracies in the outputs of an object and/or path detector), the paths and bounding shapes may still be used as the DNN is modeled such that the DNN acts as a filter for noise while performing object lane assignment. For example, in cases where the detected paths and/or bounding shapes fed to the DNN are inaccurate, the DNN still has access to the sensor data to try to filter out the noise. This formulation removes the significant burden of having to understand path and bounding shape information from just the sensor data alone (as in prior approaches) by relying outputs of object and/or path detectors that have been separately trained for detecting paths and bounding shapes. As a result, the DNN is able to focus solely on performing accurate obstacle to path assignment.

Training a Neural Network for Obstacle to Path Assignment

[0026] With reference to FIG. 1, FIG. 1 is an example data flow diagram corresponding to a process 100 for training a neural network to compute outputs usable for assigning obstacles to paths, in accordance with some embodiments of the present disclosure. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces, functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. In some embodiments, the systems, methods, and processes described herein may be executed using similar components, features, and/or functionality to those of example autonomous vehicle 900 of FIGS. 9A-9D, example computing device 1000 of FIG. 10, and/or example data center 1100 of FIG. 11.

[0027] At a high level, the process 100 may include one or more machine learning model(s) 104 receiving one or more inputs, such as sensor data 102, rasterized paths 116, and/or rasterized objects 118, and generating one or more outputs, such as one or more output channels 106. In some examples, when used for training, the sensor data 102 may be referred to as training data. Although the sensor data 102 is primarily discussed with respect to image data representative of images, this is not intended to be limiting, and the sensor data 102 may include other types of sensor data used for obstacle to path assignments, such as LiDAR data, SONAR

data, RADAR data, and/or the like—e.g., as generated by one or more sensors of the ego-machine 900 (FIGS. 9A-9D).

[0028] The process 100 may include generating and/or receiving sensor data 102 from one or more sensors of an ego-machine 900. The sensor data 102 may be received, as a non-limiting example, from one or more sensors of an autonomous or semi-autonomous machine (e.g., ego-machine 900 of FIGS. 9A-9D as described herein). The sensor data 102 may include, without limitation, sensor data 102 from any of the sensors of the ego-machine 900 including, for example and with reference to FIGS. 9A-9D, RADAR sensor(s) 960, ultrasonic sensor(s) 962, LIDAR sensor(s) 964, stereo camera(s) 968, wide-view camera(s) 970 (e.g., fisheye cameras), infrared camera(s) 972, surround camera(s) 974 (e.g., 360 degree cameras), long-range and/or mid-range camera(s) 998, and/or other sensor types. As another example, the sensor data 102 may include virtual (e.g., simulated or augmented) sensor data generated from any number of sensors of a virtual machine, vehicle, or other virtual object in a virtual (e.g., testing) environment. In such an example, the virtual sensors may correspond to a virtual vehicle or other virtual object in a realistically simulated two- or three-dimensional environment (e.g., used for testing, training, and/or validating neural network performance), and the virtual sensor data may represent sensor data captured by the virtual sensors within the simulated or virtual environment. As such, by using the virtual sensor data, the machine learning model(s) 104 described herein may be tested, trained, and/or validated using simulated data from a simulated environment and/or augmented real-world data, which may allow for testing more extreme scenarios outside of a real-world environment where such tests may be less safe.

[0029] In some embodiments, the sensor data 102 may include image data representing an image(s), image data representing a video (e.g., snapshots of video), and/or sensor data representing representations of sensory fields of sensors (e.g., depth maps for LiDAR sensors, a value graph for ultrasonic sensors, etc.). Where the sensor data 102 includes image data, any type of image data format may be used, such as, for example and without limitation, compressed images such as in Joint Photographic Experts Group (JPEG) or Luminance/Chrominance (YUV) formats, compressed images as frames stemming from a compressed video format such as H.264/Advanced Video Coding (AVC) or H.265/High Efficiency Video Coding (HEVC), raw images such as originating from Red Clear Blue (RCCB), Red Clear (RCCC), or other type of imaging sensor, and/or other formats. In addition, in some examples, the sensor data 102 may be used within the process 100 without any pre-processing (e.g., in a raw or captured format), while in other examples, the sensor data 102 may undergo pre-processing (e.g., noise balancing, demosaicing, scaling, cropping, augmentation, white balancing, tone curve adjustment, etc., such as using a sensor data pre-processor (not shown)). As used herein, the sensor data 102 may reference unprocessed sensor data, pre-processed sensor data, or a combination thereof.

[0030] The sensor data 102 used for training may include original sensor data representations (e.g., as captured by one or more sensors), down-sampled representations (e.g., down-sampled images), up-sampled representations, cropped or region of interest (ROI) representations, otherwise augmented representations, and/or a combination

thereof. The machine learning model(s) **104** may be trained using the sensor data **102** as well as corresponding ground truth data that may be generated by the ground truth encoder **114**. The ground truth encoder **114** may be configured to use the sensor data **102** to generate ground truth data such as ground truth (GT) output channels **120A-120N**. In embodiments, to generate the ground truth data, the ground truth encoder **114** may use annotations from an annotator **110**, where the annotations may include labels, masks, maps, and/or the like. For example, in some embodiments, the ground truth data may include the GT output channels **120** which may each correspond to a respective path (e.g., lane) depicted in the sensor data **102**.

[0031] With reference to the annotations from the annotator **110**, the annotations may be generated within a drawing program (e.g., an annotation program), a computer aided design (CAD) program, a labeling program, a collaborative content creation platform for 3D assets, or another type of program suitable for generating the annotations and/or the GT output channels **120**, and/or may be hand drawn. In any example, the annotations and/or the GT output channels **120** may be synthetically produced (e.g., generated from computer models or renderings), real produced (e.g., designed and produced from real-world data), machine-automated (e.g., using feature analysis and learning to extract features from data and then generate labels), human annotated (e.g., labeler, or annotation expert, defines the location of the labels), and/or a combination thereof (e.g., human identifies center or origin point and dimensions of areas, machine generates polygons and/or labels for objects and/or lanes).

[0032] The annotations may include annotations or labels of bounding shapes (e.g., boxes, squares, rectangles, circles, triangles, polygons, etc.) corresponding to object locations of objects or obstacles represented by the sensor data **102**, and/or may include annotations or labels corresponding to path (e.g., lane) locations of paths represented by the sensor data **102**. In some embodiments, the bounding shapes may be hand drawn or labeled, or may be generated using an object detection algorithm (e.g., neural network, machine learning model, computer vision algorithm, etc.). In some non-limiting embodiments, the bounding shapes may be annotated using four values, which are the (x, y) coordinates of the top left vertex and the width and height of the bounding shape. In other embodiments, the annotations may include the (x, y) locations of all four vertices, or an (x, y) location of a central point of the bounding shape and height and width information of the bounding shape. The annotations of the bounding shapes may further include a label of which path the object or obstacle belongs, and/or a classification of the object or obstacle type (e.g., vehicle, pedestrian, debris, etc.). Similarly, the path(s) may be delineated using any of a variety of methods, including a path detection algorithm (e.g., neural network, machine learning model, computer vision algorithm, etc.) configured to detect path locations, hand drawing or annotating the path(s) (e.g., human labeler draws boundaries defining the path(s), places points along boundaries of the path(s) that may be connected using polylines, etc.).

[0033] In addition, in embodiments, the bounding shapes and/or the paths may be labeled with assignment information (e.g., lane assignment, such as an identification of the lane or path an object is located, etc.) and/or semantic information. This semantic information may include, for example and without limitation, a type of object (e.g., a

vehicle, pedestrian, bicyclist, etc.) associated with a bounding shape and/or a location of a path (e.g., ego-lane, left of ego-lane, right of ego-lane, etc.) with respect to an ego-machine **900**. In some non-limiting embodiments, and with respect to the path annotations/labels, the number of paths annotated or labeled may correspond to the number of paths that the machine learning model(s) **104** is trained to compute outputs for. As an example, and with respect to FIG. 3, visualization **302** includes a plurality of points **304A, 304B, 304C**, etc., that are labeled to define the boundaries of various paths (e.g., ego-path **306B**, left of ego-path **306A**, right of ego-path **306C**) in an environment. As such, in some embodiments, annotator **110** may generate annotations including the points **304**, and polylines extending between the points, to define a contained space for each path **306**.

[0034] Rasterizer **112** may use the annotations of the bounding shapes and the paths to generate rasterized images representative of rasterized objects **118** and rasterized paths **116**, respectively. The rasterized paths **116** may be represented as a channel per path (e.g., a channel per lane), such that where there are three paths (as in visualization **302** of FIG. 3), the rasterized image for the rasterized paths **116** may include a red, green, blue (RGB) image with the red channel corresponding to a first path, the green channel corresponding to a second path, and the blue channel corresponding to a third path. Where there are less than three or more than three paths, there may be a channel per lane (e.g., a rasterized image channel per lane). As such, the boundaries of each path from the annotator **110** may be used to generate the rasterized path(s) **116**. The rasterized path(s) **116** may be similar to visualization **220** of FIG. 2. For example, each rasterized path **116** may include a binary image with first pixel values for pixels that correspond to a path **206** and second pixel values for pixels that do not correspond to the path. Although illustrated as a single image visualization **220**, this is not intended to be limiting, and the rasterized paths **116** may include an image per path **206**, as described herein. For example, a first channel (or image) may correspond to lane **202A**, a second channel (or image) may correspond to lane **202B**, and a third channel (or image) may correspond to lane **202C**.

[0035] In some embodiments, where the paths are annotated (in addition to or alternatively from using a path detection algorithm), the paths may be augmented in order to generate a diversity of path lengths, shapes, distances, sizes, etc., that more closely resembles path detection algorithms. For example, in deployment (as described with respect to FIG. 6), the paths may be detected using a path detector **604**, and the path detector **604**—based on noisy data, occlusions, slopes, inclines, etc.—may have less than ideal or optimal path detections. As such, the machine learning model(s) **104** may be trained to account for varying quality path detection results. However, the annotator **110** may generate annotations and associated path detections that are ideal (or close to optimal), and if used directly, these annotations may not allow the machine learning model(s) **104** to operate effectively in deployment using data from the path detector **604**. To simulate the real-world deployment detections, the path annotations may be augmented. To do this, in embodiments, a random number generator may be used to generate—for each instance of sensor data **102** used for training—a random percentage or value corresponding to points or a number of points to be removed from the annotations. For example, where a particular path is defined

using 100 points, and the random generator generates a value of 10%, or 10 points, then 10% or ten points may be removed from the annotations, and the remaining points may be used to generate the enclosed shape corresponding to the path. In some embodiments, to emulate inaccuracy in detecting paths as the paths extend into the distance, the percentage or number of points may correspond to the top-most points (or points further from the sensor that generated the sensor data). However, this is not intended to be limiting, and the points may be removed from the top, bottom, sides, or a combination thereof without departing from the scope of the present disclosure. In some embodiments, a percentage or value corresponding to a height of edges of the paths may be generated (e.g., using a random number generator), and the percentage or value of the height may be removed (e.g., from the top, bottom, etc.). To determine the height, in embodiments, the y coordinates of the first and last vertex of each edge may be determined, and the two values may be subtracted from one another. Then, each vertex that is under the percentage or value of the height may be removed. Although primarily described herein with respect to paths or lanes, this annotation, label, or ground truth augmentation method may be used for other types regions, objects, etc., such as signs, buildings, etc.

[0036] As an example, and with respect to visualization 302A of FIG. 3, the annotator 110 may generate the points 304A, 304B, 304C, etc. to define the various paths 306A, 306B, and 306C. In some embodiments, each of the points 304 may be used to generate the paths. However, the random number generator may be used to generate a number of points, a percentage of points, or a percentage or value of the height to be removed. The visualization 302B includes the same paths from the visualization 302A after a percentage of points are removed. As illustrated, the paths 306 are shorter in the visualization 302B, after the percentage or number of points are removed from the top-most portion of the paths 306.

[0037] As another example, and with respect to visualization 310 of FIG. 3, the annotator 110 may generate the points 304D, 304E, 304F, etc. to define the various paths 306D and 306E. In some embodiments, each of the points 304 may be used to generate the paths 306. However, the random number generator may be used to generate a number of points, a percentage of points, or a percentage or value of the height to be removed. The visualization 310B includes the same paths from the visualization 310A after a percentage of points are removed. As illustrated, the paths 306 are shorter in the visualization 310B, after the percentage or number of points are removed from the top-most portion of the paths 306.

[0038] Similarly, the bounding shapes corresponding to objects or obstacles represented by the sensor data 102 may be rasterized by the rasterizer 112 to generate the rasterized objects 118. With respect to FIG. 2, the visualization 210 may include rasterized image (e.g., a binary map or image) representing the vehicles 204A, 204B, and 204C. For example, bounding shapes may be annotated (or detected using an object detection algorithm) to correspond to each vehicle 204. In some embodiments, the bounding shapes may be used directly to generate the rasterized (or binary) image of visualization 210. For example, the vehicle 204C—instead of being represented using the ellipse in visualization 210—may be represented using the bounding shape 208C. The other vehicles 204 may also be represented

using the bounding shape. As such, the pixels within the bounding shapes 208 may have a first pixel value indicative of an object being located at those pixels, and other pixels outside of the bounding shapes may have a second pixel value indicative of no object being located at those pixels.

[0039] In some embodiments, to improve performance and accuracy, the rasterized objects 118 may be generated by cropping out a portion of the bounding shapes 208. For example, with respect to the bounding shape 208C, an ellipse, circle, or other shape (e.g., centered at a center point of the bounding shape 208C) may be generated and used to crop the bounding shape 208. As such, the pixels within the ellipse, circle, or other shape may be used in the rasterized image to indicate the location of the vehicle 204C. In this way, pixels at the edges or corners of the bounding shapes that may correspond to background (e.g., the sky, the road, etc.) may be cropped out (and encoded as not corresponding to an object), while the remaining pixels that are more likely to correspond to the actual vehicle 204C may be encoded with a value indicating presence of the vehicle 204C. The vehicles 204A and 204B may be represented similarly in the rasterized images or binary maps, which may provide a more accurate depiction of the object locations for the machine learning model(s) 104 when processing the data.

[0040] During training, the machine learning model(s) 104 may take as input the sensor data 102 (e.g., an image, a point cloud, a range image, etc.), the rasterized paths 116, and the rasterized objects 118 and may compute outputs including the one or more output channels 106. The one or more output channels 106 may include, in non-limiting embodiments, one output channel 106 per path (e.g., lane, pathway, walkway, defined region, etc.) and may indicate which pixels for the respective path are associated with or correspond to a location of an object. For example, each pixel in each output channel may have a confidence, probability, or other indicator of whether an object is located at that pixel. This information may then be used in deployment, as described herein with respect to FIG. 6, to compare to computed bounding shapes to determine lane assignments for detected objects.

[0041] The output channels 106 may then be compared to the GT output channels 120A-120N using training engine 108. For example, one or more loss functions may be used to compare the output channels 106 to the GT output channels 120, and inaccurate predictions may be penalized such that parameters (e.g., weight and/or biases) of the machine learning model(s) 104 are updated. This process may be repeated for any number of instances of training data, and for any number of epochs, until an acceptable or desired level of accuracy is reached. Once the machine learning model(s) 104 has reached a desired level of accuracy, the machine learning model(s) 104 may be used in deployment (e.g., such as in process 600 of FIG. 6).

[0042] Although examples are described herein with respect to using deep neural networks (DNNs), and specifically convolutional neural networks (CNNs), as the machine learning model(s) 104, this is not intended to be limiting. For example, and without limitation, the machine learning model(s) 104 may include any type of machine learning model, such as a machine learning model(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g.,

auto-encoders, convolutional, recurrent, perceptrons, long/short term memory/LSTM, Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), object and/or lane detection algorithms, computer vision algorithms, and/or other types of machine learning models.

[0043] As an example, such as where the machine learning model(s) **104** include a convolutional neural network (CNN), the machine learning model(s) **104** may include any number of layers. One or more of the layers may include an input layer. The input layer may hold values associated with the sensor data **102** (e.g., before or after post-processing). For example, when the sensor data **102** is an image, the input layer may hold values representative of the raw pixel values of the image(s) as a volume (e.g., a width, a height, and color channels (e.g., RGB), such as $32 \times 32 \times 3$).

[0044] One or more layers may include convolutional layers. The convolutional layers may compute the output of neurons that are connected to local regions in an input layer, each neuron computing a dot product between their weights and a small region they are connected to in the input volume. A result of the convolutional layers may be another volume, with one of the dimensions based on the number of filters applied (e.g., the width, the height, and the number of filters, such as $32 \times 32 \times 12$, if 12 were the number of filters).

[0045] One or more of the layers may include a rectified linear unit (ReLU) layer. The ReLU layer(s) may apply an elementwise activation function, such as the max (0, x), thresholding at zero, for example. The resulting volume of a ReLU layer may be the same as the volume of the input of the ReLU layer.

[0046] One or more of the layers may include a pooling layer. The pooling layer may perform a down sampling operation along the spatial dimensions (e.g., the height and the width), which may result in a smaller volume than the input of the pooling layer (e.g., $16 \times 16 \times 12$ from the $32 \times 32 \times 12$ input volume).

[0047] One or more of the layers may include one or more fully connected layer(s). Each neuron in the fully connected layer(s) may be connected to each of the neurons in the previous volume. The fully connected layer may compute class scores, and the resulting volume may be (for example and without limitation) $1 \times 1 \times \text{number of classes}$. In some examples, the CNN may include a fully connected layer(s) such that the output of one or more of the layers of the CNN may be provided as input to a fully connected layer(s) of the CNN. In some examples, one or more convolutional streams may be implemented by the machine learning model(s) **104**, and some or all of the convolutional streams may include a respective fully connected layer(s).

[0048] In some non-limiting embodiments, the machine learning model(s) **104** may include a series of convolutional and max pooling layers to facilitate image feature extraction, followed by multi-scale dilated convolutional and up-sampling layers to facilitate global context feature extraction.

[0049] Although input layers, convolutional layers, pooling layers, ReLU layers, and fully connected layers are discussed herein with respect to the machine learning model(s) **104**, this is not intended to be limiting. For example, additional or alternative layers may be used in the machine learning model(s) **104**, such as normalization layers, Soft-Max layers, and/or other layer types.

[0050] In embodiments where the machine learning model(s) **104** includes a CNN, different orders and/or numbers of

the layers of the CNN may be used depending on the embodiment. In other words, the order and number of layers of the machine learning model(s) **104** is not limited to any one architecture.

[0051] In addition, some of the layers may include parameters (e.g., weights and/or biases), such as the convolutional layers and the fully connected layers, while others may not, such as the ReLU layers and pooling layers. In some examples, the parameters may be learned by the machine learning model(s) **104** during training. Further, some of the layers may include additional hyper-parameters (e.g., learning rate, stride, epochs, etc.), such as the convolutional layers, the fully connected layers, and the pooling layers, while other layers may not, such as the ReLU layers. The parameters and hyper-parameters are not to be limited and may differ depending on the embodiment.

[0052] Now referring to FIGS. 4 and 5, each block of methods **400** and **500**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. The methods **400** and **500** may also be embodied as computer-usable instructions stored on computer storage media. The methods **400** and **500** may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, methods **400** and **500** are described, by way of example, with respect to the process **100** of FIG. 1. However, these methods **400** and **500** may additionally or alternatively be executed within any one process and/or by any one system, or any combination of processes and systems, including, but not limited to, those described herein.

[0053] With reference to FIG. 4, FIG. 4 is a flow diagram showing a method **400** for augmenting training data annotations to simulate live perception, in accordance with some embodiments of the present disclosure. The method **400**, at block **B402**, includes receiving annotation data representative of a set of points defining a shape in an instance of sensor data. For example, the annotator **110** may be used to generate points that define a shape (e.g., a path, a lane line, a lane, a linear shape, an enclosed shape, etc.). The points may be used as vertices or connection points for generating polylines. For example, where the points correspond to path boundary points, the points may be connected to generate a confined shape that may be used by the rasterizer **112** to generate the rasterized paths **116**.

[0054] The method **400**, at block **B404**, includes generating a value corresponding to a subset of points from the set of points to be removed. For example, a percentage, a number, an amount, etc. may be generated or determined (e.g., using a random number generator) that corresponds to the set of points. This number, percentage, amount, etc. may be used to determine points to be removed from the set of points, such that a reduced set of points remains. Although described as a value corresponding to a number of points to be removed, this is not intended to be limiting, and the value may instead correspond to a percentage, amount, etc., of points that should remain. In some embodiments, the value may correspond to a percentage of the height of edges corresponding to the paths to be removed, or a value corresponding to an amount of the height to be removed.

[0055] The method **400**, at block **B406**, includes removing the subset of points from the set of points to generate a reduced set of points. For example, the subset of points may be removed from the points, such that a smaller number of points remain (in instances where the value generated is non-zero). In some embodiments, such as where the points correspond to a lane line, a lane, etc., the points may be removed starting at the top (e.g., a location furthest from the sensor) and working downward toward the bottom (e.g., a location closest to the sensor). In this way, the removed points more closely correspond to sensor data errors most commonly experienced in deployment, where features further from the sensor are more likely to be occluded, not captured, etc. In other examples, however, the points may be removed starting from the top, the bottom, a side, a random location, randomly, etc., without departing from the scope of the present disclosure.

[0056] The method **400**, at block **B408**, includes generating a shape using the reduced set of points. For example, where the points correspond to a path shape, the reduced set of points may be used to generate polylines that define the path shape. Similarly, where the points correspond to lane or road boundary lines, the reduced set of points may be used to generate polylines defining the lane or road boundary lines. This process may similarly be used for any type of feature in the environment, such as dynamic or static objects, lanes, road signs, lane lanes, road boundary lines, buildings, etc.

[0057] The method **400**, a block **B410**, includes generating at least one of ground truth data for training a machine learning model or input data for the machine learning model using the shape. For example, ground truth data (e.g., GT output channels **120**) may be generated using the shape or inputs (e.g., rasterized paths **116**, rasterized objects **118**, etc.) may be generated using the shape.

[0058] Now referring to FIG. 5, FIG. 5 is a flow diagram illustrating a method for training a neural network to compute outputs usable for assigning obstacles to paths, in accordance with some embodiments of the present disclosure. The method **500**, at block **B502**, includes receiving annotation data indicative of one or more bounding shape locations corresponding to one or more objects and one or more annotations corresponding to path locations of one or more paths. For example, bounding shapes corresponding to objects or obstacles represented by the sensor data **102** and/or points, polylines, and/or other annotations used to define boundaries of paths represented by the sensor data may be generated and/or received.

[0059] The method **500**, at block **B504**, includes generating, based at least in part on the annotation data, one or more first images indicative of object locations and one or more second images indicative of path locations. For example, the rasterizer **112** may generate rasterized paths **116** and the rasterized objects **118** that may be used as input to the machine learning model(s) **104**.

[0060] The method **500**, at block **B506**, includes generating, based at least in part on the annotation data, ground truth output data indicative of one or more object locations within the one or more paths. For example, GT output channels **120** may be generated (e.g., one per path), where each GT output channel includes confidences, probabilities, etc. (e.g., per pixel) that an object is located in the path.

[0061] The method **500**, at block **B508**, includes, during training of a machine learning model, using the one or more

first images and the one or more second images as inputs to the machine learning model and the ground truth output data as ground truth. For example, during the training of the machine learning model(s) **104**, the rasterized paths **116** and the rasterized objects **118** may be used as input to the machine learning model(s) **104** (in addition to the sensor data **102**), and the GT output channels **120** may be used as ground truth by the training engine **108**.

Obstacle to Path Assignment Using a Neural Network

[0062] Referring now to FIG. 6, FIG. 6 is a data flow diagram illustrating an example process **600** for using a neural network to aid in the assignment of obstacles to paths, in accordance with some embodiments of the present disclosure. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces, functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. In some embodiments, the systems, methods, and processes described herein may be executed using similar components, features, and/or functionality to those of example autonomous vehicle **900** of FIGS. 9A-9D, example computing device **1000** of FIG. 10, and/or example data center **1100** of FIG. 11.

[0063] The sensor data **102** may be similar to that described herein at least with respect to FIG. 1. During deployment, the sensor data **102** may be applied to the machine learning model(s) **104**—e.g., that has been trained according to the process **100**—to compute the output channel(s) **106**. As such, the machine learning model(s) **104** may output one or more of the output channels **106** as described in more detail at least with respect to the output channels **106** in FIG. 1. In embodiments, for each detected path, the machine learning model(s) **104** may compute a separate output channel **106**, and these output channel(s) **106** may be used to determine obstacle to path assignments **608** for each detected object or obstacle. In other embodiments, however, there may be any number of output channels **106**, and the one or more paths may be represented by each output channel **106**, without departing from the scope of the present disclosure.

[0064] In deployment, the inputs to the machine learning model(s) **104** may be generated using an object detector **602A** and/or a path detector **604**. For example, in the process **100** of FIG. 1, the rasterized path(s) **116** and the rasterized object(s) **118** were generated using annotations from an annotator **110**. In deployment, however, the bounding shapes and path shapes or locations may be determined using the object detector **602A** and the path detector **604**, respectively. The object detector **602A** (and **602B**) and/or the path detector **604** may include any type of detection algorithm, such as a computer vision algorithm, a machine learning model(s), a neural network, etc. For example, the object detector **602A** (and **602B**) and/or the path detector **604** may include a machine learning model(s) using linear regression, logistic

regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, long/short term memory/LSTM, Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), object and/or lane detection algorithms, computer vision algorithms, and/or other types of machine learning models.

[0065] As such, the sensor data **102** (and/or other sensor data **102**) may be used by the object detector **602A** to generate bounding shapes corresponding to detected objects or obstacles. These detected bounding shapes may then be used by the rasterizer **112** to generate the rasterized object(s) **118** (e.g., rasterized images, binary maps, etc.) for use as input to the machine learning model(s) **104**. As described with respect to FIG. 1, the bounding shapes may be cropped (e.g., using an ellipse, circle, other shape, etc.) prior to using the resulting cropped shape for the rasterized object(s) **118**. Similarly, the sensor data **102** (and/or other sensor data **102**) may be used by the path detector **604** to generate path shapes and/or lines defining boundaries of paths. These detected path shapes may then be used by the rasterizer **112** to generate the rasterized path(s) **116** (e.g., rasterized images, binary maps, etc.) for use as input to the machine learning model(s) **104**. In some embodiments, as described herein, the rasterized paths **116** may include one input per path (e.g., lane). Although described as using annotations to generate the rasterized paths **116** and the rasterized objects **118** during training (FIG. 1), this is not intended to be limiting, and in some embodiments, the rasterized paths **116** and/or the rasterized objects **118** may be generated during training using an object detector **602A** and/or a path detector **604**.

[0066] As an example, and with respect to FIG. 7, visualization **700** may represent an image represented using the sensor data **102** and corresponding object detections from the object detector **602A** and path detections from the path detector **604**. For example, the object detector **602A** may compute the locations and shapes of bounding shapes **704A** and **704B** corresponding to the two vehicles depicted in the image, and the path detector **604** may compute the locations and shapes of the paths **702A** and **702B**—e.g., using polylines **706A-1** and **706A-2** for path **702A** and polylines **706B-1** and **706B-2** for path **702B**. Although only illustrated using polylines for sides of the paths **702**, this is not intended to be limiting, and the path detector **604** may detect the paths using top polylines, bottom polylines, and/or other polylines, and/or may compute the path locations and/or shapes using other methods (e.g., bounding shapes, etc.). The rasterizer **112** may then generate a rasterized path(s) **116** as represented in visualization **710** and the rasterized path(s) **116** as represented in visualization **720**. Although illustrated as a single image visualization **720**, this is not intended to be limiting, and the rasterized paths **116** may include an image per path **702**, as described herein. For example, a first channel (or image) may correspond to path **702A** and a second channel (or image) may correspond to path **702B**.

[0067] In some embodiments, one or more of the output channels **106** may undergo up-sampling and/or down-sampling. For example, the output channel(s) **106** may be generated at a first spatial resolution (e.g., a down-sampled spatial resolution) that is different from a second spatial resolution corresponding to the input spatial resolution of

the sensor data **102**. As such, the up-sampling may decode the data from the first spatial resolution of the output channel(s) **106** and convert or translate the data to the second spatial resolution of the sensor data **102**. In some examples, bi-linear up-sampling may be applied to each of the output channels **106** to recover the original resolution of the sensor data **102** and to generate corresponding up-sampled output channels **106**. As a result, the processing speeds of the machine learning model(s) **104** may be increased—e.g., due to processing reduced spatial resolution data—while simultaneously preserving compute resources for other tasks of the system. In addition, by increasing processing speeds, the run-time of the system may be decreased thereby enabling real-time or near real-time deployment of the process **600** within the system (e.g., the ego-machine **900**).

[0068] The output channel(s) **106** may be applied to a decoder **606** to generate obstacle to path assignments **608**. In some examples, the decoder **606** may use one or more post-processing algorithms (e.g., temporal smoothing, up-sampling, thresholding, clustering, mapping, etc.) on the output channel(s) **106** prior to using the output channel(s) **106** to compute the obstacle to path assignment **608**. The decoder **606** may compare bounding shapes computed using the object detector **602B** to each of the output channels **106** in order to determine a path assignment for each object or obstacle. For example, for a bounding shape of a detected object, the bounding shape may be compared to a first output channel **106** corresponding to an ego-lane and a second output channel corresponding to a right of ego-lane. If the pixel confidences are high within the bounding shape for the first output channel **106** and not as high or lower for the second output channel **106**, the object associated with the bounding shape may be assigned to the ego-lane. In addition, classification information from the object detector **602B** may be used by the decoder **606** to assign not only the path to the object or obstacle, but also to include the classification information. As such, the obstacle to path assignment **608** may include a location, pose, and/or size of the obstacle or object, the path (e.g., lane) that the obstacle or object is located (e.g., relative to the ego-machine **900**), and/or the classification of the object or obstacle. This information may then be used by an autonomous or semi-autonomous machine drive stack (“drive stack”) **610** to perform one or more operations for the ego-machine **900**.

[0069] For example, the drive stack **610** may include a sensor manager, a perception component(s) (e.g., corresponding to a perception layer of the drive stack **610**), a world model manager, a planning component(s) (e.g., corresponding to a planning layer of the drive stack **610**), a control component(s) (e.g., corresponding to a control layer of the drive stack **610**), an obstacle avoidance component(s) (e.g., corresponding to an obstacle or collision avoidance layer of the drive stack **610**), an actuation component(s) (e.g., corresponding to an actuation layer of the drive stack **610**), and/or other components corresponding to additional and/or alternative layers of the drive stack **610**.

[0070] The sensor manager may manage and/or abstract the sensor data **102** from the sensors of the ego-machine **900**. For example, and with reference to FIG. 9C, the sensor data **102** may be generated (e.g., perpetually, at intervals, based on certain conditions, etc.) by RADAR sensor(s) **960**, ultrasonic sensor(s) **962**, LIDAR sensor(s) **964**, stereo camera(s) **968**, wide-view camera(s) **970**, infrared camera(s)

972, surround camera(s) 974, long range and/or mid-range camera(s) 998, and/or other sensor types.

[0071] The sensor manager may receive the sensor data 102 from the sensors in different formats (e.g., sensors of the same type, such as image sensors of cameras, may output sensor data in different formats), and may be configured to convert the different formats to a uniform format (e.g., for each sensor of the same type). As a result, other components, features, and/or functionality of the ego-machine 900 may use the uniform format, thereby simplifying processing of the sensor data 102. In some examples, the sensor manager may use a uniform format to apply control back to the sensors of the vehicle 900, such as to set frame rates or to perform gain control. The sensor manager may also update sensor packets or communications corresponding to the sensor data with timestamps to help inform processing of the sensor data by various components, features, and functionality of an autonomous vehicle control system.

[0072] A world model manager may be used to generate, update, and/or define a world model. The world model manager may use information generated by and received from the perception component(s) of the drive stack 610 (e.g., the locations of paths, objects, obstacles, etc., classification information, assignment information, etc.). The perception component(s) may include an obstacle perceiver, a path perceiver, a wait perceiver, a map perceiver, and/or other perception component(s). For example, the world model may be defined, at least in part, based on affordances for obstacles, paths, and wait conditions that can be perceived in real-time or near real-time by the obstacle perceiver, the path perceiver, the wait perceiver, and/or the map perceiver. The world model manager may continually update the world model based on newly generated and/or received inputs (e.g., data) from the obstacle perceiver, the path perceiver, the wait perceiver, the map perceiver, and/or other components of the autonomous vehicle control system.

[0073] The world model may be used to help inform a planning component(s), a control component(s), an obstacle avoidance component(s), and/or an actuation component(s) of the drive stack 610. The obstacle perceiver may perform obstacle perception that may be based on where the ego-machine 900 is allowed to drive or is capable of driving (e.g., based on the location of the drivable paths as well as object and/or obstacle locations), and how fast the ego-machine can drive without colliding with an obstacle (e.g., an object, such as a structure, entity, vehicle, etc.) that is sensed by the sensors of the ego-machine 900.

[0074] The path perceiver may perform path perception, such as by perceiving nominal paths that are available in a particular situation. In some examples, the path perceiver may further take into account lane changes for path perception. A lane graph may represent the path or paths available to the ego-machine 900, and may be as simple as a single path on a highway on-ramp. In some examples, the lane graph may include paths to a desired lane and/or may indicate available changes down the highway (or other road type), or may include nearby lanes, lane changes, forks, turns, cloverleaf interchanges, merges, and/or other information.

[0075] The wait perceiver may be responsible to determining constraints on the ego-machine 900 as a result of rules, conventions, and/or practical considerations. For example, the rules, conventions, and/or practical considerations may be in relation to traffic lights, multi-way stops,

yields, merges, toll booths, gates, police or other emergency personnel, road workers, stopped buses or other vehicles, one-way bridge arbitrations, ferry entrances, etc. Thus, the wait perceiver may be used to identify potential obstacles and implement one or more controls (e.g., slowing down, coming to a stop, etc.) that may not have been possible relying solely on the obstacle perceiver.

[0076] The map perceiver may include a mechanism by which behaviors are discerned, and in some examples, to determine specific examples of what conventions are applied at a particular locale. For example, the map perceiver may determine, from data representing prior drives or trips, that at a certain intersection there are no U-turns between certain hours, that an electronic sign showing directionality of lanes changes depending on the time of day, that two traffic lights in close proximity (e.g., barely offset from one another) are associated with different roads, that in Rhode Island, the first car waiting to make a left turn at traffic light breaks the law by turning before oncoming traffic when the light turns green, and/or other information. The map perceiver may inform the ego-machine 900 of static or stationary infrastructure objects and obstacles. The map perceiver may also generate information for the wait perceiver and/or the path perceiver, for example, such as to determine which light at an intersection has to be green for the ego-machine 900 to take a particular path.

[0077] In some examples, information from the map perceiver may be sent, transmitted, and/or provided to server(s) (e.g., to a map manager of server(s) 978 of FIG. 9D), and information from the server(s) may be sent, transmitted, and/or provided to the map perceiver and/or a localization manager of the ego-machine 900. The map manager may include a cloud mapping application that is remotely located from the ego-machine 900 and accessible by the ego-machine 900 over one or more network(s). For example, the map perceiver and/or the localization manager of the ego-machine 900 may communicate with the map manager and/or one or more other components or features of the server(s) to inform the map perceiver and/or the localization manager of past and present drives or trips of the ego-machine 900, as well as past and present drives or trips of other vehicles. The map manager may provide mapping outputs (e.g., map data) that may be localized by the localization manager based on a particular location of the ego-machine 900, and the localized mapping outputs may be used by the world model manager to generate and/or update the world model.

[0078] The planning component(s) may include a route planner, a lane planner, a behavior planner, and a behavior selector, among other components, features, and/or functionality. The route planner may use the information from the map perceiver, the map manager, and/or the localization manager, among other information, to generate a planned path that may consist of GNSS waypoints (e.g., GPS waypoints), 3D world coordinates (e.g., Cartesian, polar, etc.) that indicate coordinates relative to an origin point on the ego-machine 900, etc. The waypoints may be representative of a specific distance into the future for the ego-machine 900, such as a number of city blocks, a number of kilometers, a number of feet, a number of inches, a number of miles, etc., that may be used as a target for the lane planner.

[0079] The lane planner may use the lane graph (e.g., the lane graph from the path perceiver), object poses within the lane graph (e.g., according to the localization manager),

and/or a target point and direction at the distance into the future from the route planner as inputs. The target point and direction may be mapped to the best matching drivable point and direction in the lane graph (e.g., based on GNSS and/or compass direction). A graph search algorithm may then be executed on the lane graph from a current edge in the lane graph to find the shortest path to the target point.

[0080] The behavior planner may determine the feasibility of basic behaviors of the ego-machine **900**, such as staying in the lane or changing lanes left or right, so that the feasible behaviors may be matched up with the most desired behaviors output from the lane planner. For example, if the desired behavior is determined to not be safe and/or available, a default behavior may be selected instead (e.g., default behavior may be to stay in lane when desired behavior or changing lanes is not safe).

[0081] The control component(s) may follow a trajectory or path (lateral and longitudinal) that has been received from the behavior selector (e.g., based on the path geometry(ies) and/or the class labels) of the planning component(s) as closely as possible and within the capabilities of the ego-machine **900**. The control component(s) may use tight feedback to handle unplanned events or behaviors that are not modeled and/or anything that causes discrepancies from the ideal (e.g., unexpected delay). In some examples, the control component(s) may use a forward prediction model that takes control as an input variable, and produces predictions that may be compared with the desired state (e.g., compared with the desired lateral and longitudinal path requested by the planning component(s)). The control(s) that minimize discrepancy may be determined.

[0082] Although the planning component(s) and the control component(s) are described separately, this is not intended to be limiting. For example, in some embodiments, the delineation between the planning component(s) and the control component(s) may not be precisely defined. As such, at least some of the components, features, and/or functionality attributed to the planning component(s) may be associated with the control component(s), and vice versa. This may also hold true for any of the separately illustrated components of the drive stack **610**.

[0083] The obstacle avoidance component(s) may aid the autonomous ego-machine **900** in avoiding collisions with objects (e.g., moving and stationary objects). The obstacle avoidance component(s) may include a computational mechanism at a “primal level” of obstacle avoidance, and may act as a “survival brain” or “reptile brain” for the ego-machine **900**. In some examples, the obstacle avoidance component(s) may be used independently of components, features, and/or functionality of the ego-machine **900** that is required to obey traffic rules and drive courteously. In such examples, the obstacle avoidance component(s) may ignore traffic laws, rules of the road, and courteous driving norms in order to ensure that collisions do not occur between the ego-machine **900** and any objects. As such, the obstacle avoidance layer may be a separate layer from the rules of the road layer, and the obstacle avoidance layer may ensure that the ego-machine **900** is only performing safe actions from an obstacle avoidance standpoint. The rules of the road layer, on the other hand, may ensure that vehicle obeys traffic laws and conventions, and observes lawful and conventional right of way (as described herein).

[0084] In some examples, the drivable paths as defined by the path geometries and/or the path type(s) corresponding to

each of the path geometries may be used by the obstacle avoidance component(s) in determining controls or actions to take. For example, the drivable paths may provide an indication to the obstacle avoidance component(s) of where the ego-machine **900** may maneuver without striking any objects, structures, and/or the like, or at least where no static structures may exist.

[0085] In non-limiting embodiments, the obstacle avoidance component(s) may be implemented as a separate, discrete feature of the ego-machine **900**. For example, the obstacle avoidance component(s) may operate separately (e.g., in parallel with, prior to, and/or after) the planning layer, the control layer, the actuation layer, and/or other layers of the drive stack **610**.

[0086] Now referring to FIG. **8**, each block of method **800**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. The method **800** may also be embodied as computer-usable instructions stored on computer storage media. The method **800** may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product or service, to name a few. In addition, method **800** is described, by way of example, with respect to the process **600** of FIG. **6**. However, this method **800** may additionally or alternatively be executed within any one process and/or by any one system, or any combination of processes and systems, including, but not limited to, those described herein.

[0087] With reference to FIG. **8**, FIG. **8** is a flow diagram showing a method **800** of using a neural network to aid in the assignment of obstacles to paths, in accordance with some embodiments of the present disclosure. The method **800**, at block **B802**, includes generating, from a sensor data representation, a first input representative of one or more path locations of one or more paths in the sensor data representation and a second input representative of object locations in the sensor data representation. For example, the rasterized path(s) **116** and the rasterized object(s) **118** may be generated using the sensor data **102**, the object detector **602A**, the path detector **604**, and/or the rasterizer **112**.

[0088] The method **800**, at block **B804**, includes computing, using a neural network and based at least in part on the sensor data representation, the first input, and the second input, one or more outputs indicative of the object locations within the one or more paths. For example, the machine learning model(s) **104** may process the sensor data **102**, the rasterized path(s) **116**, and/or the rasterized object(s) **118** to generate the output channel(s) **106**.

[0089] The method **800**, at block **B806**, includes, for at least one detected object, comparing a bounding shape corresponding to the at least one detected object to at least one output of the one or more outputs. For example, the decoder **606** may compare bounding shapes corresponding to detected objects from the object detector **602B** to the output channel(s) **106**.

[0090] The method **800**, at block **B808**, includes, based at least in part on the comparison, determining a path of the one or more paths that the at least one detected object is located. For example, the decoder **606** may determine the obstacle to path assignments **608** based on comparing the bounding shapes to the output channel(s) **106**.

[0091] The method 800, at block B810, includes performing one or more control operations for a machine based at least in part on the at least one detected object being located in the path. For example, the drive stack 610 of the ego-machine 900 may use the obstacle to path assignments 608 to perform one or more operations.

[0092] Compared to previous approaches, the systems and methods of the present disclosure make use of detected paths and bounding shapes, and the machine learning model(s) 104 learns to filter noise such that knowledge gained by detected paths and bounding shapes may be used, while the machine learning model(s) 104 remains less prone to detection errors. In addition, the task of obstacle to path assignment if formulated as a binary classification problem—whereas previous approaches formulated the problem as a semantic classification problem—which allows the approach to be easily extendable to any number of paths and obstacles, and allows the process to operate more seamlessly with the rest of the autonomous navigation system.

Example Autonomous Vehicle

[0093] FIG. 9A is an illustration of an example autonomous vehicle 900, in accordance with some embodiments of the present disclosure. The autonomous vehicle 900 (alternatively referred to herein as the “vehicle 900”) may include, without limitation, a passenger vehicle, such as a car, a truck, a bus, a first responder vehicle, a shuttle, an electric or motorized bicycle, a motorcycle, a fire truck, a police vehicle, an ambulance, a boat, a construction vehicle, an underwater craft, a drone, a vehicle coupled to a trailer, and/or another type of vehicle (e.g., that is unmanned and/or that accommodates one or more passengers). Autonomous vehicles are generally described in terms of automation levels, defined by the National Highway Traffic Safety Administration (NHTSA), a division of the US Department of Transportation, and the Society of Automotive Engineers (SAE) “Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles” (Standard No. J3016-201806, published on Jun. 15, 2018, Standard No. J3016-201609, published on Sep. 30, 2016, and previous and future versions of this standard). The vehicle 900 may be capable of functionality in accordance with one or more of Level 3–Level 5 of the autonomous driving levels. The vehicle 900 may be capable of functionality in accordance with one or more of Level 1–Level 5 of the autonomous driving levels. For example, the vehicle 900 may be capable of driver assistance (Level 1), partial automation (Level 2), conditional automation (Level 3), high automation (Level 4), and/or full automation (Level 5), depending on the embodiment. The term “autonomous,” as used herein, may include any and/or all types of autonomy for the vehicle 900 or other machine, such as being fully autonomous, being highly autonomous, being conditionally autonomous, being partially autonomous, providing assistive autonomy, being semi-autonomous, being primarily autonomous, or other designation.

[0094] The vehicle 900 may include components such as a chassis, a vehicle body, wheels (e.g., 2, 4, 6, 8, 18, etc.), tires, axles, and other components of a vehicle. The vehicle 900 may include a propulsion system 950, such as an internal combustion engine, hybrid electric power plant, an all-electric engine, and/or another propulsion system type. The propulsion system 950 may be connected to a drive train of the vehicle 900, which may include a transmission, to

enable the propulsion of the vehicle 900. The propulsion system 950 may be controlled in response to receiving signals from the throttle/accelerator 952.

[0095] A steering system 954, which may include a steering wheel, may be used to steer the vehicle 900 (e.g., along a desired path or route) when the propulsion system 950 is operating (e.g., when the vehicle is in motion). The steering system 954 may receive signals from a steering actuator 956. The steering wheel may be optional for full automation (Level 5) functionality.

[0096] The brake sensor system 946 may be used to operate the vehicle brakes in response to receiving signals from the brake actuators 948 and/or brake sensors.

[0097] Controller(s) 936, which may include one or more system on chips (SoCs) 904 (FIG. 9C) and/or GPU(s), may provide signals (e.g., representative of commands) to one or more components and/or systems of the vehicle 900. For example, the controller(s) may send signals to operate the vehicle brakes via one or more brake actuators 948, to operate the steering system 954 via one or more steering actuators 956, to operate the propulsion system 950 via one or more throttle/accelerators 952. The controller(s) 936 may include one or more onboard (e.g., integrated) computing devices (e.g., supercomputers) that process sensor signals, and output operation commands (e.g., signals representing commands) to enable autonomous driving and/or to assist a human driver in driving the vehicle 900. The controller(s) 936 may include a first controller 936 for autonomous driving functions, a second controller 936 for functional safety functions, a third controller 936 for artificial intelligence functionality (e.g., computer vision), a fourth controller 936 for infotainment functionality, a fifth controller 936 for redundancy in emergency conditions, and/or other controllers. In some examples, a single controller 936 may handle two or more of the above functionalities, two or more controllers 936 may handle a single functionality, and/or any combination thereof.

[0098] The controller(s) 936 may provide the signals for controlling one or more components and/or systems of the vehicle 900 in response to sensor data received from one or more sensors (e.g., sensor inputs). The sensor data may be received from, for example and without limitation, global navigation satellite systems sensor(s) 958 (e.g., Global Positioning System sensor(s)), RADAR sensor(s) 960, ultrasonic sensor(s) 962, LIDAR sensor(s) 964, inertial measurement unit (IMU) sensor(s) 966 (e.g., accelerometer(s), gyroscope(s), magnetic compass(es), magnetometer(s), etc.), microphone(s) 996, stereo camera(s) 968, wide-view camera(s) 970 (e.g., fisheye cameras), infrared camera(s) 972, surround camera(s) 974 (e.g., 360 degree cameras), long-range and/or mid-range camera(s) 998, speed sensor(s) 944 (e.g., for measuring the speed of the vehicle 900), vibration sensor(s) 942, steering sensor(s) 940, brake sensor(s) (e.g., as part of the brake sensor system 946), and/or other sensor types.

[0099] One or more of the controller(s) 936 may receive inputs (e.g., represented by input data) from an instrument cluster 932 of the vehicle 900 and provide outputs (e.g., represented by output data, display data, etc.) via a human-machine interface (HMI) display 934, an audible annunciator, a loudspeaker, and/or via other components of the vehicle 900. The outputs may include information such as vehicle velocity, speed, time, map data (e.g., the HD map 922 of FIG. 9C), location data (e.g., the vehicle's 900

location, such as on a map), direction, location of other vehicles (e.g., an occupancy grid), information about objects and status of objects as perceived by the controller(s) 936, etc. For example, the HMI display 934 may display information about the presence of one or more objects (e.g., a street sign, caution sign, traffic light changing, etc.), and/or information about driving maneuvers the vehicle has made, is making, or will make (e.g., changing lanes now, taking exit 34B in two miles, etc.).

[0100] The vehicle 900 further includes a network interface 924 which may use one or more wireless antenna(s) 926 and/or modem(s) to communicate over one or more networks. For example, the network interface 924 may be capable of communication over LTE, WCDMA, UMTS, GSM, CDMA2000, etc. The wireless antenna(s) 926 may also enable communication between objects in the environment (e.g., vehicles, mobile devices, etc.), using local area network(s), such as Bluetooth, Bluetooth LE, Z-Wave, Zig-Bee, etc., and/or low power wide-area network(s) (LP-WANs), such as LoRaWAN, SigFox, etc.

[0101] FIG. 9B is an example of camera locations and fields of view for the example autonomous vehicle 900 of FIG. 9A, in accordance with some embodiments of the present disclosure. The cameras and respective fields of view are one example embodiment and are not intended to be limiting. For example, additional and/or alternative cameras may be included and/or the cameras may be located at different locations on the vehicle 900.

[0102] The camera types for the cameras may include, but are not limited to, digital cameras that may be adapted for use with the components and/or systems of the vehicle 900. The camera(s) may operate at automotive safety integrity level (ASIL) B and/or at another ASIL. The camera types may be capable of any image capture rate, such as 60 frames per second (fps), 120 fps, 240 fps, etc., depending on the embodiment. The cameras may be capable of using rolling shutters, global shutters, another type of shutter, or a combination thereof. In some examples, the color filter array may include a red clear clear clear (RCCC) color filter array, a red clear clear blue (RCCB) color filter array, a red blue green clear (RBGC) color filter array, a Foveon X3 color filter array, a Bayer sensors (RGGB) color filter array, a monochrome sensor color filter array, and/or another type of color filter array. In some embodiments, clear pixel cameras, such as cameras with an RCCC, an RCCB, and/or an RBGC color filter array, may be used in an effort to increase light sensitivity.

[0103] In some examples, one or more of the camera(s) may be used to perform advanced driver assistance systems (ADAS) functions (e.g., as part of a redundant or fail-safe design). For example, a Multi-Function Mono Camera may be installed to provide functions including lane departure warning, traffic sign assist and intelligent headlamp control. One or more of the camera(s) (e.g., all of the cameras) may record and provide image data (e.g., video) simultaneously.

[0104] One or more of the cameras may be mounted in a mounting assembly, such as a custom designed (3-D printed) assembly, in order to cut out stray light and reflections from within the car (e.g., reflections from the dashboard reflected in the windshield mirrors) which may interfere with the camera's image data capture abilities. With reference to wing-mirror mounting assemblies, the wing-mirror assemblies may be custom 3-D printed so that the camera mounting plate matches the shape of the wing-mirror. In some

examples, the camera(s) may be integrated into the wing-mirror. For side-view cameras, the camera(s) may also be integrated within the four pillars at each corner of the cabin.

[0105] Cameras with a field of view that include portions of the environment in front of the vehicle 900 (e.g., front-facing cameras) may be used for surround view, to help identify forward facing paths and obstacles, as well aid in, with the help of one or more controllers 936 and/or control SoCs, providing information critical to generating an occupancy grid and/or determining the preferred vehicle paths. Front-facing cameras may be used to perform many of the same ADAS functions as LIDAR, including emergency braking, pedestrian detection, and collision avoidance. Front-facing cameras may also be used for ADAS functions and systems including Lane Departure Warnings (LDW), Autonomous Cruise Control (ACC), and/or other functions such as traffic sign recognition.

[0106] A variety of cameras may be used in a front-facing configuration, including, for example, a monocular camera platform that includes a CMOS (complementary metal oxide semiconductor) color imager. Another example may be a wide-view camera(s) 970 that may be used to perceive objects coming into view from the periphery (e.g., pedestrians, crossing traffic or bicycles). Although only one wide-view camera is illustrated in FIG. 9B, there may any number of wide-view cameras 970 on the vehicle 900. In addition, long-range camera(s) 998 (e.g., a long-view stereo camera pair) may be used for depth-based object detection, especially for objects for which a neural network has not yet been trained. The long-range camera(s) 998 may also be used for object detection and classification, as well as basic object tracking.

[0107] One or more stereo cameras 968 may also be included in a front-facing configuration. The stereo camera(s) 968 may include an integrated control unit comprising a scalable processing unit, which may provide a programmable logic (FPGA) and a multi-core micro-processor with an integrated CAN or Ethernet interface on a single chip. Such a unit may be used to generate a 3-D map of the vehicle's environment, including a distance estimate for all the points in the image. An alternative stereo camera(s) 968 may include a compact stereo vision sensor(s) that may include two camera lenses (one each on the left and right) and an image processing chip that may measure the distance from the vehicle to the target object and use the generated information (e.g., metadata) to activate the autonomous emergency braking and lane departure warning functions. Other types of stereo camera(s) 968 may be used in addition to, or alternatively from, those described herein.

[0108] Cameras with a field of view that include portions of the environment to the side of the vehicle 900 (e.g., side-view cameras) may be used for surround view, providing information used to create and update the occupancy grid, as well as to generate side impact collision warnings. For example, surround camera(s) 974 (e.g., four surround cameras 974 as illustrated in FIG. 9B) may be positioned to on the vehicle 900. The surround camera(s) 974 may include wide-view camera(s) 970, fisheye camera(s), 360 degree camera(s), and/or the like. Four example, four fisheye cameras may be positioned on the vehicle's front, rear, and sides. In an alternative arrangement, the vehicle may use three surround camera(s) 974 (e.g., left, right, and rear), and may leverage one or more other camera(s) (e.g., a forward-facing camera) as a fourth surround view camera.

[0109] Cameras with a field of view that include portions of the environment to the rear of the vehicle 900 (e.g., rear-view cameras) may be used for park assistance, surround view, rear collision warnings, and creating and updating the occupancy grid. A wide variety of cameras may be used including, but not limited to, cameras that are also suitable as a front-facing camera(s) (e.g., long-range and/or mid-range camera(s) 998, stereo camera(s) 968), infrared camera(s) 972, etc.), as described herein.

[0110] FIG. 9C is a block diagram of an example system architecture for the example autonomous vehicle 900 of FIG. 9A, in accordance with some embodiments of the present disclosure. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces, functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory.

[0111] Each of the components, features, and systems of the vehicle 900 in FIG. 9C are illustrated as being connected via bus 902. The bus 902 may include a Controller Area Network (CAN) data interface (alternatively referred to herein as a “CAN bus”). A CAN may be a network inside the vehicle 900 used to aid in control of various features and functionality of the vehicle 900, such as actuation of brakes, acceleration, braking, steering, windshield wipers, etc. A CAN bus may be configured to have dozens or even hundreds of nodes, each with its own unique identifier (e.g., a CAN ID). The CAN bus may be read to find steering wheel angle, ground speed, engine revolutions per minute (RPMs), button positions, and/or other vehicle status indicators. The CAN bus may be ASIL B compliant.

[0112] Although the bus 902 is described herein as being a CAN bus, this is not intended to be limiting. For example, in addition to, or alternatively from, the CAN bus, FlexRay and/or Ethernet may be used. Additionally, although a single line is used to represent the bus 902, this is not intended to be limiting. For example, there may be any number of busses 902, which may include one or more CAN busses, one or more FlexRay busses, one or more Ethernet busses, and/or one or more other types of busses using a different protocol. In some examples, two or more busses 902 may be used to perform different functions, and/or may be used for redundancy. For example, a first bus 902 may be used for collision avoidance functionality and a second bus 902 may be used for actuation control. In any example, each bus 902 may communicate with any of the components of the vehicle 900, and two or more busses 902 may communicate with the same components. In some examples, each SoC 904, each controller 936, and/or each computer within the vehicle may have access to the same input data (e.g., inputs from sensors of the vehicle 900), and may be connected to a common bus, such the CAN bus.

[0113] The vehicle 900 may include one or more controller(s) 936, such as those described herein with respect to FIG. 9A. The controller(s) 936 may be used for a variety of

functions. The controller(s) 936 may be coupled to any of the various other components and systems of the vehicle 900, and may be used for control of the vehicle 900, artificial intelligence of the vehicle 900, infotainment for the vehicle 900, and/or the like.

[0114] The vehicle 900 may include a system(s) on a chip (SoC) 904. The SoC 904 may include CPU(s) 906, GPU(s) 908, processor(s) 910, cache(s) 912, accelerator(s) 914, data store(s) 916, and/or other components and features not illustrated. The SoC(s) 904 may be used to control the vehicle 900 in a variety of platforms and systems. For example, the SoC(s) 904 may be combined in a system (e.g., the system of the vehicle 900) with an HD map 922 which may obtain map refreshes and/or updates via a network interface 924 from one or more servers (e.g., server(s) 978 of FIG. 9D).

[0115] The CPU(s) 906 may include a CPU cluster or CPU complex (alternatively referred to herein as a “CCPLEX”). The CPU(s) 906 may include multiple cores and/or L2 caches. For example, in some embodiments, the CPU(s) 906 may include eight cores in a coherent multi-processor configuration. In some embodiments, the CPU(s) 906 may include four dual-core clusters where each cluster has a dedicated L2 cache (e.g., a 2 MB L2 cache). The CPU(s) 906 (e.g., the CCPLEX) may be configured to support simultaneous cluster operation enabling any combination of the clusters of the CPU(s) 906 to be active at any given time.

[0116] The CPU(s) 906 may implement power management capabilities that include one or more of the following features: individual hardware blocks may be clock-gated automatically when idle to save dynamic power; each core clock may be gated when the core is not actively executing instructions due to execution of WFI/WFE instructions; each core may be independently power-gated; each core cluster may be independently clock-gated when all cores are clock-gated or power-gated; and/or each core cluster may be independently power-gated when all cores are power-gated. The CPU(s) 906 may further implement an enhanced algorithm for managing power states, where allowed power states and expected wakeup times are specified, and the hardware/microcode determines the best power state to enter for the core, cluster, and CCPLEX. The processing cores may support simplified power state entry sequences in software with the work offloaded to microcode.

[0117] The GPU(s) 908 may include an integrated GPU (alternatively referred to herein as an “iGPU”). The GPU(s) 908 may be programmable and may be efficient for parallel workloads. The GPU(s) 908, in some examples, may use an enhanced tensor instruction set. The GPU(s) 908 may include one or more streaming microprocessors, where each streaming microprocessor may include an L1 cache (e.g., an L1 cache with at least 96 KB storage capacity), and two or more of the streaming microprocessors may share an L2 cache (e.g., an L2 cache with a 512 KB storage capacity). In some embodiments, the GPU(s) 908 may include at least eight streaming microprocessors. The GPU(s) 908 may use compute application programming interface(s) (API(s)). In addition, the GPU(s) 908 may use one or more parallel computing platforms and/or programming models (e.g., NVIDIA’s CUDA).

[0118] The GPU(s) 908 may be power-optimized for best performance in automotive and embedded use cases. For example, the GPU(s) 908 may be fabricated on a Fin field-effect transistor (FinFET). However, this is not

intended to be limiting and the GPU(s) 908 may be fabricated using other semiconductor manufacturing processes. Each streaming microprocessor may incorporate a number of mixed-precision processing cores partitioned into multiple blocks. For example, and without limitation, 64 PF32 cores and 32 PF64 cores may be partitioned into four processing blocks. In such an example, each processing block may be allocated 16 FP32 cores, 8 FP64 cores, 16 INT32 cores, two mixed-precision NVIDIA TENSOR COREs for deep learning matrix arithmetic, an L0 instruction cache, a warp scheduler, a dispatch unit, and/or a 64 KB register file. In addition, the streaming microprocessors may include independent parallel integer and floating-point data paths to provide for efficient execution of workloads with a mix of computation and addressing calculations. The streaming microprocessors may include independent thread scheduling capability to enable finer-grain synchronization and cooperation between parallel threads. The streaming microprocessors may include a combined L1 data cache and shared memory unit in order to improve performance while simplifying programming.

[0119] The GPU(s) 908 may include a high bandwidth memory (HBM) and/or a 16 GB HBM2 memory subsystem to provide, in some examples, about 900 GB/second peak memory bandwidth. In some examples, in addition to, or alternatively from, the HBM memory, a synchronous graphics random-access memory (SGRAM) may be used, such as a graphics double data rate type five synchronous random-access memory (GDDR5).

[0120] The GPU(s) 908 may include unified memory technology including access counters to allow for more accurate migration of memory pages to the processor that accesses them most frequently, thereby improving efficiency for memory ranges shared between processors. In some examples, address translation services (ATS) support may be used to allow the GPU(s) 908 to access the CPU(s) 906 page tables directly. In such examples, when the GPU(s) 908 memory management unit (MMU) experiences a miss, an address translation request may be transmitted to the CPU(s) 906. In response, the CPU(s) 906 may look in its page tables for the virtual-to-physical mapping for the address and transmits the translation back to the GPU(s) 908. As such, unified memory technology may allow a single unified virtual address space for memory of both the CPU(s) 906 and the GPU(s) 908, thereby simplifying the GPU(s) 908 programming and porting of applications to the GPU(s) 908.

[0121] In addition, the GPU(s) 908 may include an access counter that may keep track of the frequency of access of the GPU(s) 908 to memory of other processors. The access counter may help ensure that memory pages are moved to the physical memory of the processor that is accessing the pages most frequently.

[0122] The SoC(s) 904 may include any number of cache(s) 912, including those described herein. For example, the cache(s) 912 may include an L3 cache that is available to both the CPU(s) 906 and the GPU(s) 908 (e.g., that is connected both the CPU(s) 906 and the GPU(s) 908). The cache(s) 912 may include a write-back cache that may keep track of states of lines, such as by using a cache coherence protocol (e.g., MEI, MESI, MSI, etc.). The L3 cache may include 4 MB or more, depending on the embodiment, although smaller cache sizes may be used.

[0123] The SoC(s) 904 may include an arithmetic logic unit(s) (ALU(s)) which may be leveraged in performing

processing with respect to any of the variety of tasks or operations of the vehicle 900—such as processing DNNs. In addition, the SoC(s) 904 may include a floating point unit(s) (FPU(s))—or other math coprocessor or numeric coprocessor types—for performing mathematical operations within the system. For example, the SoC(s) 104 may include one or more FPUs integrated as execution units within a CPU(s) 906 and/or GPU(s) 908.

[0124] The SoC(s) 904 may include one or more accelerators 914 (e.g., hardware accelerators, software accelerators, or a combination thereof). For example, the SoC(s) 904 may include a hardware acceleration cluster that may include optimized hardware accelerators and/or large on-chip memory. The large on-chip memory (e.g., 4 MB of SRAM), may enable the hardware acceleration cluster to accelerate neural networks and other calculations. The hardware acceleration cluster may be used to complement the GPU(s) 908 and to off-load some of the tasks of the GPU(s) 908 (e.g., to free up more cycles of the GPU(s) 908 for performing other tasks). As an example, the accelerator(s) 914 may be used for targeted workloads (e.g., perception, convolutional neural networks (CNNs), etc.) that are stable enough to be amenable to acceleration. The term “CNN,” as used herein, may include all types of CNNs, including region-based or regional convolutional neural networks (RCNNs) and Fast RCNNs (e.g., as used for object detection).

[0125] The accelerator(s) 914 (e.g., the hardware acceleration cluster) may include a deep learning accelerator(s) (DLA). The DLA(s) may include one or more Tensor processing units (TPUs) that may be configured to provide an additional ten trillion operations per second for deep learning applications and inferencing. The TPUs may be accelerators configured to, and optimized for, performing image processing functions (e.g., for CNNs, RCNNs, etc.). The DLA(s) may further be optimized for a specific set of neural network types and floating point operations, as well as inferencing. The design of the DLA(s) may provide more performance per millimeter than a general-purpose GPU, and vastly exceeds the performance of a CPU. The TPU(s) may perform several functions, including a single-instance convolution function, supporting, for example, INT8, INT16, and FP16 data types for both features and weights, as well as post-processor functions.

[0126] The DLA(s) may quickly and efficiently execute neural networks, especially CNNs, on processed or unprocessed data for any of a variety of functions, including, for example and without limitation: a CNN for object identification and detection using data from camera sensors; a CNN for distance estimation using data from camera sensors; a CNN for emergency vehicle detection and identification and detection using data from microphones; a CNN for facial recognition and vehicle owner identification using data from camera sensors; and/or a CNN for security and/or safety related events.

[0127] The DLA(s) may perform any function of the GPU(s) 908, and by using an inference accelerator, for example, a designer may target either the DLA(s) or the GPU(s) 908 for any function. For example, the designer may focus processing of CNNs and floating point operations on the DLA(s) and leave other functions to the GPU(s) 908 and/or other accelerator(s) 914.

[0128] The accelerator(s) 914 (e.g., the hardware acceleration cluster) may include a programmable vision accel-

erator(s) (PVA), which may alternatively be referred to herein as a computer vision accelerator. The PVA(s) may be designed and configured to accelerate computer vision algorithms for the advanced driver assistance systems (ADAS), autonomous driving, and/or augmented reality (AR) and/or virtual reality (VR) applications. The PVA(s) may provide a balance between performance and flexibility. For example, each PVA(s) may include, for example and without limitation, any number of reduced instruction set computer (RISC) cores, direct memory access (DMA), and/or any number of vector processors.

[0129] The RISC cores may interact with image sensors (e.g., the image sensors of any of the cameras described herein), image signal processor(s), and/or the like. Each of the RISC cores may include any amount of memory. The RISC cores may use any of a number of protocols, depending on the embodiment. In some examples, the RISC cores may execute a real-time operating system (RTOS). The RISC cores may be implemented using one or more integrated circuit devices, application specific integrated circuits (ASICs), and/or memory devices. For example, the RISC cores may include an instruction cache and/or a tightly coupled RAM.

[0130] The DMA may enable components of the PVA(s) to access the system memory independently of the CPU(s) 906. The DMA may support any number of features used to provide optimization to the PVA including, but not limited to, supporting multi-dimensional addressing and/or circular addressing. In some examples, the DMA may support up to six or more dimensions of addressing, which may include block width, block height, block depth, horizontal block stepping, vertical block stepping, and/or depth stepping.

[0131] The vector processors may be programmable processors that may be designed to efficiently and flexibly execute programming for computer vision algorithms and provide signal processing capabilities. In some examples, the PVA may include a PVA core and two vector processing subsystem partitions. The PVA core may include a processor subsystem, DMA engine(s) (e.g., two DMA engines), and/or other peripherals. The vector processing subsystem may operate as the primary processing engine of the PVA, and may include a vector processing unit (VPU), an instruction cache, and/or vector memory (e.g., VMEM). A VPU core may include a digital signal processor such as, for example, a single instruction, multiple data (SIMD), very long instruction word (VLIW) digital signal processor. The combination of the SIMD and VLIW may enhance throughput and speed.

[0132] Each of the vector processors may include an instruction cache and may be coupled to dedicated memory. As a result, in some examples, each of the vector processors may be configured to execute independently of the other vector processors. In other examples, the vector processors that are included in a particular PVA may be configured to employ data parallelism. For example, in some embodiments, the plurality of vector processors included in a single PVA may execute the same computer vision algorithm, but on different regions of an image. In other examples, the vector processors included in a particular PVA may simultaneously execute different computer vision algorithms, on the same image, or even execute different algorithms on sequential images or portions of an image. Among other things, any number of PVAs may be included in the hardware acceleration cluster and any number of vector processors may be included in each of the PVAs. In addition, the

PVA(s) may include additional error correcting code (ECC) memory, to enhance overall system safety.

[0133] The accelerator(s) 914 (e.g., the hardware acceleration cluster) may include a computer vision network on-chip and SRAM, for providing a high-bandwidth, low latency SRAM for the accelerator(s) 914. In some examples, the on-chip memory may include at least 4 MB SRAM, consisting of, for example and without limitation, eight field-configurable memory blocks, that may be accessible by both the PVA and the DLA. Each pair of memory blocks may include an advanced peripheral bus (APB) interface, configuration circuitry, a controller, and a multiplexer. Any type of memory may be used. The PVA and DLA may access the memory via a backbone that provides the PVA and DLA with high-speed access to memory. The backbone may include a computer vision network on-chip that interconnects the PVA and the DLA to the memory (e.g., using the APB).

[0134] The computer vision network on-chip may include an interface that determines, before transmission of any control signal/address/data, that both the PVA and the DLA provide ready and valid signals. Such an interface may provide for separate phases and separate channels for transmitting control signals/addresses/data, as well as burst-type communications for continuous data transfer. This type of interface may comply with ISO 26262 or IEC 61508 standards, although other standards and protocols may be used.

[0135] In some examples, the SoC(s) 904 may include a real-time ray-tracing hardware accelerator, such as described in U.S. patent application Ser. No. 16/101,232, filed on Aug. 10, 2018. The real-time ray-tracing hardware accelerator may be used to quickly and efficiently determine the positions and extents of objects (e.g., within a world model), to generate real-time visualization simulations, for RADAR signal interpretation, for sound propagation synthesis and/or analysis, for simulation of SONAR systems, for general wave propagation simulation, for comparison to LIDAR data for purposes of localization and/or other functions, and/or for other uses. In some embodiments, one or more tree traversal units (TTUs) may be used for executing one or more ray-tracing related operations.

[0136] The accelerator(s) 914 (e.g., the hardware acceleration cluster) have a wide array of uses for autonomous driving. The PVA may be a programmable vision accelerator that may be used for key processing stages in ADAS and autonomous vehicles. The PVA's capabilities are a good match for algorithmic domains needing predictable processing, at low power and low latency. In other words, the PVA performs well on semi-dense or dense regular computation, even on small data sets, which need predictable run-times with low latency and low power. Thus, in the context of platforms for autonomous vehicles, the PVAs are designed to run classic computer vision algorithms, as they are efficient at object detection and operating on integer math.

[0137] For example, according to one embodiment of the technology, the PVA is used to perform computer stereo vision. A semi-global matching-based algorithm may be used in some examples, although this is not intended to be limiting. Many applications for Level 3-5 autonomous driving require motion estimation/stereo matching on-the-fly (e.g., structure from motion, pedestrian recognition, lane detection, etc.). The PVA may perform computer stereo vision function on inputs from two monocular cameras.

[0138] In some examples, the PVA may be used to perform dense optical flow. According to process raw RADAR data (e.g., using a 4D Fast Fourier Transform) to provide Processed RADAR. In other examples, the PVA is used for time of flight depth processing, by processing raw time of flight data to provide processed time of flight data, for example.

[0139] The DLA may be used to run any type of network to enhance control and driving safety, including for example, a neural network that outputs a measure of confidence for each object detection. Such a confidence value may be interpreted as a probability, or as providing a relative “weight” of each detection compared to other detections. This confidence value enables the system to make further decisions regarding which detections should be considered as true positive detections rather than false positive detections. For example, the system may set a threshold value for the confidence and consider only the detections exceeding the threshold value as true positive detections. In an automatic emergency braking (AEB) system, false positive detections would cause the vehicle to automatically perform emergency braking, which is obviously undesirable. Therefore, only the most confident detections should be considered as triggers for AEB. The DLA may run a neural network for regressing the confidence value. The neural network may take as its input at least some subset of parameters, such as bounding box dimensions, ground plane estimate obtained (e.g. from another subsystem), inertial measurement unit (IMU) sensor **966** output that correlates with the vehicle **900** orientation, distance, 3D location estimates of the object obtained from the neural network and/or other sensors (e.g., LIDAR sensor(s) **964** or RADAR sensor(s) **960**), among others.

[0140] The SoC(s) **904** may include data store(s) **916** (e.g., memory). The data store(s) **916** may be on-chip memory of the SoC(s) **904**, which may store neural networks to be executed on the GPU and/or the DLA. In some examples, the data store(s) **916** may be large enough in capacity to store multiple instances of neural networks for redundancy and safety. The data store(s) **912** may comprise L2 or L3 cache(s) **912**. Reference to the data store(s) **916** may include reference to the memory associated with the PVA, DLA, and/or other accelerator(s) **914**, as described herein.

[0141] The SoC(s) **904** may include one or more processor(s) **910** (e.g., embedded processors). The processor(s) **910** may include a boot and power management processor that may be a dedicated processor and subsystem to handle boot power and management functions and related security enforcement. The boot and power management processor may be a part of the SoC(s) **904** boot sequence and may provide runtime power management services. The boot power and management processor may provide clock and voltage programming, assistance in system low power state transitions, management of SoC(s) **904** thermal and temperature sensors, and/or management of the SoC(s) **904** power states. Each temperature sensor may be implemented as a ring-oscillator whose output frequency is proportional to temperature, and the SoC(s) **904** may use the ring-oscillators to detect temperatures of the CPU(s) **906**, GPU(s) **908**, and/or accelerator(s) **914**. If temperatures are determined to exceed a threshold, the boot and power management processor may enter a temperature fault routine and put the SoC(s) **904** into a lower power state and/or put the vehicle **900** into a chauffeur to safe stop mode (e.g., bring the vehicle **900** to a safe stop).

[0142] The processor(s) **910** may further include a set of embedded processors that may serve as an audio processing engine. The audio processing engine may be an audio subsystem that enables full hardware core support for multi-channel audio over multiple interfaces, and a broad and flexible range of audio I/O interfaces. In some examples, the audio processing engine is a dedicated processor core with a digital signal processor with dedicated RAM.

[0143] The processor(s) **910** may further include an always on processor engine that may provide necessary hardware features to support low power sensor management and wake use cases. The always on processor engine may include a processor core, a tightly coupled RAM, supporting peripherals (e.g., timers and interrupt controllers), various I/O controller peripherals, and routing logic.

[0144] The processor(s) **910** may further include a safety cluster engine that includes a dedicated processor subsystem to handle safety management for automotive applications. The safety cluster engine may include two or more processor cores, a tightly coupled RAM, support peripherals (e.g., timers, an interrupt controller, etc.), and/or routing logic. In a safety mode, the two or more cores may operate in a lockstep mode and function as a single core with comparison logic to detect any differences between their operations.

[0145] The processor(s) **910** may further include a real-time camera engine that may include a dedicated processor subsystem for handling real-time camera management.

[0146] The processor(s) **910** may further include a high-dynamic range signal processor that may include an image signal processor that is a hardware engine that is part of the camera processing pipeline.

[0147] The processor(s) **910** may include a video image compositor that may be a processing block (e.g., implemented on a microprocessor) that implements video post-processing functions needed by a video playback application to produce the final image for the player window. The video image compositor may perform lens distortion correction on wide-view camera(s) **970**, surround camera(s) **974**, and/or on in-cabin monitoring camera sensors. In-cabin monitoring camera sensor is preferably monitored by a neural network running on another instance of the Advanced SoC, configured to identify in cabin events and respond accordingly. An in-cabin system may perform lip reading to activate cellular service and place a phone call, dictate emails, change the vehicle’s destination, activate or change the vehicle’s infotainment system and settings, or provide voice-activated web surfing. Certain functions are available to the driver only when the vehicle is operating in an autonomous mode, and are disabled otherwise.

[0148] The video image compositor may include enhanced temporal noise reduction for both spatial and temporal noise reduction. For example, where motion occurs in a video, the noise reduction weights spatial information appropriately, decreasing the weight of information provided by adjacent frames. Where an image or portion of an image does not include motion, the temporal noise reduction performed by the video image compositor may use information from the previous image to reduce noise in the current image.

[0149] The video image compositor may also be configured to perform stereo rectification on input stereo lens frames. The video image compositor may further be used for user interface composition when the operating system desktop is in use, and the GPU(s) **908** is not required to

continuously render new surfaces. Even when the GPU(s) 908 is powered on and active doing 3D rendering, the video image compositor may be used to offload the GPU(s) 908 to improve performance and responsiveness.

[0150] The SoC(s) 904 may further include a mobile industry processor interface (MIPI) camera serial interface for receiving video and input from cameras, a high-speed interface, and/or a video input block that may be used for camera and related pixel input functions. The SoC(s) 904 may further include an input/output controller(s) that may be controlled by software and may be used for receiving I/O signals that are uncommitted to a specific role.

[0151] The SoC(s) 904 may further include a broad range of peripheral interfaces to enable communication with peripherals, audio codecs, power management, and/or other devices. The SoC(s) 904 may be used to process data from cameras (e.g., connected over Gigabit Multimedia Serial Link and Ethernet), sensors (e.g., LIDAR sensor(s) 964, RADAR sensor(s) 960, etc. that may be connected over Ethernet), data from bus 902 (e.g., speed of vehicle 900, steering wheel position, etc.), data from GNSS sensor(s) 958 (e.g., connected over Ethernet or CAN bus). The SoC(s) 904 may further include dedicated high-performance mass storage controllers that may include their own DMA engines, and that may be used to free the CPU(s) 906 from routine data management tasks.

[0152] The SoC(s) 904 may be an end-to-end platform with a flexible architecture that spans automation levels 3-5, thereby providing a comprehensive functional safety architecture that leverages and makes efficient use of computer vision and ADAS techniques for diversity and redundancy, provides a platform for a flexible, reliable driving software stack, along with deep learning tools. The SoC(s) 904 may be faster, more reliable, and even more energy-efficient and space-efficient than conventional systems. For example, the accelerator(s) 914, when combined with the CPU(s) 906, the GPU(s) 908, and the data store(s) 916, may provide for a fast, efficient platform for level 3-5 autonomous vehicles.

[0153] The technology thus provides capabilities and functionality that cannot be achieved by conventional systems. For example, computer vision algorithms may be executed on CPUs, which may be configured using high-level programming language, such as the C programming language, to execute a wide variety of processing algorithms across a wide variety of visual data. However, CPUs are oftentimes unable to meet the performance requirements of many computer vision applications, such as those related to execution time and power consumption, for example. In particular, many CPUs are unable to execute complex object detection algorithms in real-time, which is a requirement of in-vehicle ADAS applications, and a requirement for practical Level 3-5 autonomous vehicles.

[0154] In contrast to conventional systems, by providing a CPU complex, GPU complex, and a hardware acceleration cluster, the technology described herein allows for multiple neural networks to be performed simultaneously and/or sequentially, and for the results to be combined together to enable Level 3-5 autonomous driving functionality. For example, a CNN executing on the DLA or dGPU (e.g., the GPU(s) 920) may include a text and word recognition, allowing the supercomputer to read and understand traffic signs, including signs for which the neural network has not been specifically trained. The DLA may further include a neural network that is able to identify, interpret, and provides

semantic understanding of the sign, and to pass that semantic understanding to the path planning modules running on the CPU Complex.

[0155] As another example, multiple neural networks may be run simultaneously, as is required for Level 3, 4, or 5 driving. For example, a warning sign consisting of “Caution: flashing lights indicate icy conditions,” along with an electric light, may be independently or collectively interpreted by several neural networks. The sign itself may be identified as a traffic sign by a first deployed neural network (e.g., a neural network that has been trained), the text “Flashing lights indicate icy conditions” may be interpreted by a second deployed neural network, which informs the vehicle’s path planning software (preferably executing on the CPU Complex) that when flashing lights are detected, icy conditions exist. The flashing light may be identified by operating a third deployed neural network over multiple frames, informing the vehicle’s path-planning software of the presence (or absence) of flashing lights. All three neural networks may run simultaneously, such as within the DLA and/or on the GPU(s) 908.

[0156] In some examples, a CNN for facial recognition and vehicle owner identification may use data from camera sensors to identify the presence of an authorized driver and/or owner of the vehicle 900. The always on sensor processing engine may be used to unlock the vehicle when the owner approaches the driver door and turn on the lights, and, in security mode, to disable the vehicle when the owner leaves the vehicle. In this way, the SoC(s) 904 provide for security against theft and/or carjacking.

[0157] In another example, a CNN for emergency vehicle detection and identification may use data from microphones 996 to detect and identify emergency vehicle sirens. In contrast to conventional systems, that use general classifiers to detect sirens and manually extract features, the SoC(s) 904 use the CNN for classifying environmental and urban sounds, as well as classifying visual data. In a preferred embodiment, the CNN running on the DLA is trained to identify the relative closing speed of the emergency vehicle (e.g., by using the Doppler Effect). The CNN may also be trained to identify emergency vehicles specific to the local area in which the vehicle is operating, as identified by GNSS sensor(s) 958. Thus, for example, when operating in Europe the CNN will seek to detect European sirens, and when in the United States the CNN will seek to identify only North American sirens. Once an emergency vehicle is detected, a control program may be used to execute an emergency vehicle safety routine, slowing the vehicle, pulling over to the side of the road, parking the vehicle, and/or idling the vehicle, with the assistance of ultrasonic sensors 962, until the emergency vehicle(s) passes.

[0158] The vehicle may include a CPU(s) 918 (e.g., discrete CPU(s), or dCPU(s)), that may be coupled to the SoC(s) 904 via a high-speed interconnect (e.g., PCIe). The CPU(s) 918 may include an X86 processor, for example. The CPU(s) 918 may be used to perform any of a variety of functions, including arbitrating potentially inconsistent results between ADAS sensors and the SoC(s) 904, and/or monitoring the status and health of the controller(s) 936 and/or infotainment SoC 930, for example.

[0159] The vehicle 900 may include a GPU(s) 920 (e.g., discrete GPU(s), or dGPU(s)), that may be coupled to the SoC(s) 904 via a high-speed interconnect (e.g., NVIDIA’s NVLINK). The GPU(s) 920 may provide additional artifi-

cial intelligence functionality, such as by executing redundant and/or different neural networks, and may be used to train and/or update neural networks based on input (e.g., sensor data) from sensors of the vehicle 900.

[0160] The vehicle 900 may further include the network interface 924 which may include one or more wireless antennas 926 (e.g., one or more wireless antennas for different communication protocols, such as a cellular antenna, a Bluetooth antenna, etc.). The network interface 924 may be used to enable wireless connectivity over the Internet with the cloud (e.g., with the server(s) 978 and/or other network devices), with other vehicles, and/or with computing devices (e.g., client devices of passengers). To communicate with other vehicles, a direct link may be established between the two vehicles and/or an indirect link may be established (e.g., across networks and over the Internet). Direct links may be provided using a vehicle-to-vehicle communication link. The vehicle-to-vehicle communication link may provide the vehicle 900 information about vehicles in proximity to the vehicle 900 (e.g., vehicles in front of, on the side of, and/or behind the vehicle 900). This functionality may be part of a cooperative adaptive cruise control functionality of the vehicle 900.

[0161] The network interface 924 may include a SoC that provides modulation and demodulation functionality and enables the controller(s) 936 to communicate over wireless networks. The network interface 924 may include a radio frequency front-end for up-conversion from baseband to radio frequency, and down conversion from radio frequency to baseband. The frequency conversions may be performed through well-known processes, and/or may be performed using super-heterodyne processes. In some examples, the radio frequency front end functionality may be provided by a separate chip. The network interface may include wireless functionality for communicating over LTE, WCDMA, UMTS, GSM, CDMA2000, Bluetooth, Bluetooth LE, Wi-Fi, Z-Wave, ZigBee, LoRaWAN, and/or other wireless protocols.

[0162] The vehicle 900 may further include data store(s) 928 which may include off-chip (e.g., off the SoC(s) 904) storage. The data store(s) 928 may include one or more storage elements including RAM, SRAM, DRAM, VRAM, Flash, hard disks, and/or other components and/or devices that may store at least one bit of data.

[0163] The vehicle 900 may further include GNSS sensor(s) 958. The GNSS sensor(s) 958 (e.g., GPS, assisted GPS sensors, differential GPS (DGPS) sensors, etc.), to assist in mapping, perception, occupancy grid generation, and/or path planning functions. Any number of GNSS sensor(s) 958 may be used, including, for example and without limitation, a GPS using a USB connector with an Ethernet to Serial (RS-232) bridge.

[0164] The vehicle 900 may further include RADAR sensor(s) 960. The RADAR sensor(s) 960 may be used by the vehicle 900 for long-range vehicle detection, even in darkness and/or severe weather conditions. RADAR functional safety levels may be ASIL B. The RADAR sensor(s) 960 may use the CAN and/or the bus 902 (e.g., to transmit data generated by the RADAR sensor(s) 960) for control and to access object tracking data, with access to Ethernet to access raw data in some examples. A wide variety of RADAR sensor types may be used. For example, and without limitation, the RADAR sensor(s) 960 may be suit-

able for front, rear, and side RADAR use. In some example, Pulse Doppler RADAR sensor(s) are used.

[0165] The RADAR sensor(s) 960 may include different configurations, such as long range with narrow field of view, short range with wide field of view, short range side coverage, etc. In some examples, long-range RADAR may be used for adaptive cruise control functionality. The long-range RADAR systems may provide a broad field of view realized by two or more independent scans, such as within a 250 m range. The RADAR sensor(s) 960 may help in distinguishing between static and moving objects, and may be used by ADAS systems for emergency brake assist and forward collision warning. Long-range RADAR sensors may include monostatic multimodal RADAR with multiple (e.g., six or more) fixed RADAR antennae and a high-speed CAN and FlexRay interface. In an example with six antennae, the central four antennae may create a focused beam pattern, designed to record the vehicle's 900 surroundings at higher speeds with minimal interference from traffic in adjacent lanes. The other two antennae may expand the field of view, making it possible to quickly detect vehicles entering or leaving the vehicle's 900 lane.

[0166] Mid-range RADAR systems may include, as an example, a range of up to 960 m (front) or 80 m (rear), and a field of view of up to 42 degrees (front) or 950 degrees (rear). Short-range RADAR systems may include, without limitation, RADAR sensors designed to be installed at both ends of the rear bumper. When installed at both ends of the rear bumper, such a RADAR sensor systems may create two beams that constantly monitor the blind spot in the rear and next to the vehicle.

[0167] Short-range RADAR systems may be used in an ADAS system for blind spot detection and/or lane change assist.

[0168] The vehicle 900 may further include ultrasonic sensor(s) 962. The ultrasonic sensor(s) 962, which may be positioned at the front, back, and/or the sides of the vehicle 900, may be used for park assist and/or to create and update an occupancy grid. A wide variety of ultrasonic sensor(s) 962 may be used, and different ultrasonic sensor(s) 962 may be used for different ranges of detection (e.g., 2.5 m, 4 m). The ultrasonic sensor(s) 962 may operate at functional safety levels of ASIL B.

[0169] The vehicle 900 may include LIDAR sensor(s) 964. The LIDAR sensor(s) 964 may be used for object and pedestrian detection, emergency braking, collision avoidance, and/or other functions. The LIDAR sensor(s) 964 may be functional safety level ASIL B. In some examples, the vehicle 900 may include multiple LIDAR sensors 964 (e.g., two, four, six, etc.) that may use Ethernet (e.g., to provide data to a Gigabit Ethernet switch).

[0170] In some examples, the LIDAR sensor(s) 964 may be capable of providing a list of objects and their distances for a 360-degree field of view. Commercially available LIDAR sensor(s) 964 may have an advertised range of approximately 900 m, with an accuracy of 2 cm-3 cm, and with support for a 900 Mbps Ethernet connection, for example. In some examples, one or more non-protruding LIDAR sensors 964 may be used. In such examples, the LIDAR sensor(s) 964 may be implemented as a small device that may be embedded into the front, rear, sides, and/or corners of the vehicle 900. The LIDAR sensor(s) 964, in such examples, may provide up to a 120-degree horizontal and 35-degree vertical field-of-view, with a 200 m range

even for low-reflectivity objects. Front-mounted LIDAR sensor(s) 964 may be configured for a horizontal field of view between 45 degrees and 135 degrees.

[0171] In some examples, LIDAR technologies, such as 3D flash LIDAR, may also be used. 3D Flash LIDAR uses a flash of a laser as a transmission source, to illuminate vehicle surroundings up to approximately 200 m. A flash LIDAR unit includes a receptor, which records the laser pulse transit time and the reflected light on each pixel, which in turn corresponds to the range from the vehicle to the objects. Flash LIDAR may allow for highly accurate and distortion-free images of the surroundings to be generated with every laser flash. In some examples, four flash LIDAR sensors may be deployed, one at each side of the vehicle 900. Available 3D flash LIDAR systems include a solid-state 3D staring array LIDAR camera with no moving parts other than a fan (e.g., a non-scanning LIDAR device). The flash LIDAR device may use a 5 nanosecond class I (eye-safe) laser pulse per frame and may capture the reflected laser light in the form of 3D range point clouds and co-registered intensity data. By using flash LIDAR, and because flash LIDAR is a solid-state device with no moving parts, the LIDAR sensor(s) 964 may be less susceptible to motion blur, vibration, and/or shock.

[0172] The vehicle may further include IMU sensor(s) 966. The IMU sensor(s) 966 may be located at a center of the rear axle of the vehicle 900, in some examples. The IMU sensor(s) 966 may include, for example and without limitation, an accelerometer(s), a magnetometer(s), a gyroscope (s), a magnetic compass(es), and/or other sensor types. In some examples, such as in six-axis applications, the IMU sensor(s) 966 may include accelerometers and gyroscopes, while in nine-axis applications, the IMU sensor(s) 966 may include accelerometers, gyroscopes, and magnetometers.

[0173] In some embodiments, the IMU sensor(s) 966 may be implemented as a miniature, high performance GPS-Aided Inertial Navigation System (GPS/INS) that combines micro-electro-mechanical systems (MEMS) inertial sensors, a high-sensitivity GPS receiver, and advanced Kalman filtering algorithms to provide estimates of position, velocity, and attitude. As such, in some examples, the IMU sensor(s) 966 may enable the vehicle 900 to estimate heading without requiring input from a magnetic sensor by directly observing and correlating the changes in velocity from GPS to the IMU sensor(s) 966. In some examples, the IMU sensor(s) 966 and the GNSS sensor(s) 958 may be combined in a single integrated unit.

[0174] The vehicle may include microphone(s) 996 placed in and/or around the vehicle 900. The microphone(s) 996 may be used for emergency vehicle detection and identification, among other things.

[0175] The vehicle may further include any number of camera types, including stereo camera(s) 968, wide-view camera(s) 970, infrared camera(s) 972, surround camera(s) 974, long-range and/or mid-range camera(s) 998, and/or other camera types. The cameras may be used to capture image data around an entire periphery of the vehicle 900. The types of cameras used depends on the embodiments and requirements for the vehicle 900, and any combination of camera types may be used to provide the necessary coverage around the vehicle 900. In addition, the number of cameras may differ depending on the embodiment. For example, the vehicle may include six cameras, seven cameras, ten cameras, twelve cameras, and/or another number of cameras.

The cameras may support, as an example and without limitation, Gigabit Multimedia Serial Link (GMSL) and/or Gigabit Ethernet. Each of the camera(s) is described with more detail herein with respect to FIG. 9A and FIG. 9B.

[0176] The vehicle 900 may further include vibration sensor(s) 942. The vibration sensor(s) 942 may measure vibrations of components of the vehicle, such as the axle(s). For example, changes in vibrations may indicate a change in road surfaces. In another example, when two or more vibration sensors 942 are used, the differences between the vibrations may be used to determine friction or slippage of the road surface (e.g., when the difference in vibration is between a power-driven axle and a freely rotating axle).

[0177] The vehicle 900 may include an ADAS system 938. The ADAS system 938 may include a SoC, in some examples. The ADAS system 938 may include autonomous/adaptive/automatic cruise control (ACC), cooperative adaptive cruise control (CACC), forward crash warning (FCW), automatic emergency braking (AEB), lane departure warnings (LDW), lane keep assist (LKA), blind spot warning (BSW), rear cross-traffic warning (RCTW), collision warning systems (CWS), lane centering (LC), and/or other features and functionality.

[0178] The ACC systems may use RADAR sensor(s) 960, LIDAR sensor(s) 964, and/or a camera(s). The ACC systems may include longitudinal ACC and/or lateral ACC. Longitudinal ACC monitors and controls the distance to the vehicle immediately ahead of the vehicle 900 and automatically adjust the vehicle speed to maintain a safe distance from vehicles ahead. Lateral ACC performs distance keeping, and advises the vehicle 900 to change lanes when necessary. Lateral ACC is related to other ADAS applications such as LCA and CWS.

[0179] CACC uses information from other vehicles that may be received via the network interface 924 and/or the wireless antenna(s) 926 from other vehicles via a wireless link, or indirectly, over a network connection (e.g., over the Internet). Direct links may be provided by a vehicle-to-vehicle (V2V) communication link, while indirect links may be infrastructure-to-vehicle (I2V) communication link. In general, the V2V communication concept provides information about the immediately preceding vehicles (e.g., vehicles immediately ahead of and in the same lane as the vehicle 900), while the I2V communication concept provides information about traffic further ahead. CACC systems may include either or both I2V and V2V information sources. Given the information of the vehicles ahead of the vehicle 900, CACC may be more reliable and it has potential to improve traffic flow smoothness and reduce congestion on the road.

[0180] FCW systems are designed to alert the driver to a hazard, so that the driver may take corrective action. FCW systems use a front-facing camera and/or RADAR sensor(s) 960, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component. FCW systems may provide a warning, such as in the form of a sound, visual warning, vibration and/or a quick brake pulse.

[0181] AEB systems detect an impending forward collision with another vehicle or other object, and may automatically apply the brakes if the driver does not take corrective action within a specified time or distance parameter. AEB systems may use front-facing camera(s) and/or RADAR sensor(s) 960, coupled to a dedicated processor,

DSP, FPGA, and/or ASIC. When the AEB system detects a hazard, it typically first alerts the driver to take corrective action to avoid the collision and, if the driver does not take corrective action, the AEB system may automatically apply the brakes in an effort to prevent, or at least mitigate, the impact of the predicted collision. AEB systems, may include techniques such as dynamic brake support and/or crash imminent braking.

[0182] LDW systems provide visual, audible, and/or tactile warnings, such as steering wheel or seat vibrations, to alert the driver when the vehicle 900 crosses lane markings. A LDW system does not activate when the driver indicates an intentional lane departure, by activating a turn signal. LDW systems may use front-side facing cameras, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0183] LKA systems are a variation of LDW systems. LKA systems provide steering input or braking to correct the vehicle 900 if the vehicle 900 starts to exit the lane.

[0184] BSW systems detects and warn the driver of vehicles in an automobile's blind spot. BSW systems may provide a visual, audible, and/or tactile alert to indicate that merging or changing lanes is unsafe. The system may provide an additional warning when the driver uses a turn signal. BSW systems may use rear-side facing camera(s) and/or RADAR sensor(s) 960, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0185] RCTW systems may provide visual, audible, and/or tactile notification when an object is detected outside the rear-camera range when the vehicle 900 is backing up. Some RCTW systems include AEB to ensure that the vehicle brakes are applied to avoid a crash. RCTW systems may use one or more rear-facing RADAR sensor(s) 960, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0186] Conventional ADAS systems may be prone to false positive results which may be annoying and distracting to a driver, but typically are not catastrophic, because the ADAS systems alert the driver and allow the driver to decide whether a safety condition truly exists and act accordingly. However, in an autonomous vehicle 900, the vehicle 900 itself must, in the case of conflicting results, decide whether to heed the result from a primary computer or a secondary computer (e.g., a first controller 936 or a second controller 936). For example, in some embodiments, the ADAS system 938 may be a backup and/or secondary computer for providing perception information to a backup computer rationality module. The backup computer rationality monitor may run a redundant diverse software on hardware components to detect faults in perception and dynamic driving tasks. Outputs from the ADAS system 938 may be provided to a supervisory MCU. If outputs from the primary computer and the secondary computer conflict, the supervisory MCU must determine how to reconcile the conflict to ensure safe operation.

[0187] In some examples, the primary computer may be configured to provide the supervisory MCU with a confidence score, indicating the primary computer's confidence in the chosen result. If the confidence score exceeds a threshold, the supervisory MCU may follow the primary

computer's direction, regardless of whether the secondary computer provides a conflicting or inconsistent result. Where the confidence score does not meet the threshold, and where the primary and secondary computer indicate different results (e.g., the conflict), the supervisory MCU may arbitrate between the computers to determine the appropriate outcome.

[0188] The supervisory MCU may be configured to run a neural network(s) that is trained and configured to determine, based on outputs from the primary computer and the secondary computer, conditions under which the secondary computer provides false alarms. Thus, the neural network(s) in the supervisory MCU may learn when the secondary computer's output may be trusted, and when it cannot. For example, when the secondary computer is a RADAR-based FCW system, a neural network(s) in the supervisory MCU may learn when the FCW system is identifying metallic objects that are not, in fact, hazards, such as a drainage grate or manhole cover that triggers an alarm. Similarly, when the secondary computer is a camera-based LDW system, a neural network in the supervisory MCU may learn to override the LDW when bicyclists or pedestrians are present and a lane departure is, in fact, the safest maneuver. In embodiments that include a neural network(s) running on the supervisory MCU, the supervisory MCU may include at least one of a DLA or GPU suitable for running the neural network(s) with associated memory. In preferred embodiments, the supervisory MCU may comprise and/or be included as a component of the SoC(s) 904.

[0189] In other examples, ADAS system 938 may include a secondary computer that performs ADAS functionality using traditional rules of computer vision. As such, the secondary computer may use classic computer vision rules (if-then), and the presence of a neural network(s) in the supervisory MCU may improve reliability, safety and performance. For example, the diverse implementation and intentional non-identity makes the overall system more fault-tolerant, especially to faults caused by software (or software-hardware interface) functionality. For example, if there is a software bug or error in the software running on the primary computer, and the non-identical software code running on the secondary computer provides the same overall result, the supervisory MCU may have greater confidence that the overall result is correct, and the bug in software or hardware on primary computer is not causing material error.

[0190] In some examples, the output of the ADAS system 938 may be fed into the primary computer's perception block and/or the primary computer's dynamic driving task block. For example, if the ADAS system 938 indicates a forward crash warning due to an object immediately ahead, the perception block may use this information when identifying objects. In other examples, the secondary computer may have its own neural network which is trained and thus reduces the risk of false positives, as described herein.

[0191] The vehicle 900 may further include the infotainment SoC 930 (e.g., an in-vehicle infotainment system (IVI)). Although illustrated and described as a SoC, the infotainment system may not be a SoC, and may include two or more discrete components. The infotainment SoC 930 may include a combination of hardware and software that may be used to provide audio (e.g., music, a personal digital assistant, navigational instructions, news, radio, etc.), video (e.g., TV, movies, streaming, etc.), phone (e.g., hands-free

calling), network connectivity (e.g., LTE, Wi-Fi, etc.), and/or information services (e.g., navigation systems, rear-parking assistance, a radio data system, vehicle related information such as fuel level, total distance covered, brake fuel level, oil level, door open/close, air filter information, etc.) to the vehicle 900. For example, the infotainment SoC 930 may include radios, disk players, navigation systems, video players, USB and Bluetooth connectivity, carputers, in-car entertainment, Wi-Fi, steering wheel audio controls, hands free voice control, a heads-up display (HUD), an HMI display 934, a telematics device, a control panel (e.g., for controlling and/or interacting with various components, features, and/or systems), and/or other components. The infotainment SoC 930 may further be used to provide information (e.g., visual and/or audible) to a user(s) of the vehicle, such as information from the ADAS system 938, autonomous driving information such as planned vehicle maneuvers, trajectories, surrounding environment information (e.g., intersection information, vehicle information, road information, etc.), and/or other information.

[0192] The infotainment SoC 930 may include GPU functionality. The infotainment SoC 930 may communicate over the bus 902 (e.g., CAN bus, Ethernet, etc.) with other devices, systems, and/or components of the vehicle 900. In some examples, the infotainment SoC 930 may be coupled to a supervisory MCU such that the GPU of the infotainment system may perform some self-driving functions in the event that the primary controller(s) 936 (e.g., the primary and/or backup computers of the vehicle 900) fail. In such an example, the infotainment SoC 930 may put the vehicle 900 into a chauffeur to safe stop mode, as described herein.

[0193] The vehicle 900 may further include an instrument cluster 932 (e.g., a digital dash, an electronic instrument cluster, a digital instrument panel, etc.). The instrument cluster 932 may include a controller and/or supercomputer (e.g., a discrete controller or supercomputer). The instrument cluster 932 may include a set of instrumentation such as a speedometer, fuel level, oil pressure, tachometer, odometer, turn indicators, gearshift position indicator, seat belt warning light(s), parking-brake warning light(s), engine-malfunction light(s), airbag (SRS) system information, lighting controls, safety system controls, navigation information, etc. In some examples, information may be displayed and/or shared among the infotainment SoC 930 and the instrument cluster 932. In other words, the instrument cluster 932 may be included as part of the infotainment SoC 930, or vice versa.

[0194] FIG. 9D is a system diagram for communication between cloud-based server(s) and the example autonomous vehicle 900 of FIG. 9A, in accordance with some embodiments of the present disclosure. The system 976 may include server(s) 978, network(s) 990, and vehicles, including the vehicle 900. The server(s) 978 may include a plurality of GPUs 984(A)-984(H) (collectively referred to herein as GPUs 984), PCIe switches 982(A)-982(H) (collectively referred to herein as PCIe switches 982), and/or CPUs 980(A)-980(B) (collectively referred to herein as CPUs 980). The GPUs 984, the CPUs 980, and the PCIe switches may be interconnected with high-speed interconnects such as, for example and without limitation, NVLink interfaces 988 developed by NVIDIA and/or PCIe connections 986. In some examples, the GPUs 984 are connected via NVLink and/or NVSwitch SoC and the GPUs 984 and the PCIe switches 982 are connected via PCIe interconnects.

Although eight GPUs 984, two CPUs 980, and two PCIe switches are illustrated, this is not intended to be limiting. Depending on the embodiment, each of the server(s) 978 may include any number of GPUs 984, CPUs 980, and/or PCIe switches. For example, the server(s) 978 may each include eight, sixteen, thirty-two, and/or more GPUs 984.

[0195] The server(s) 978 may receive, over the network(s) 990 and from the vehicles, image data representative of images showing unexpected or changed road conditions, such as recently commenced road-work. The server(s) 978 may transmit, over the network(s) 990 and to the vehicles, neural networks 992, updated neural networks 992, and/or map information 994, including information regarding traffic and road conditions. The updates to the map information 994 may include updates for the HD map 922, such as information regarding construction sites, potholes, detours, flooding, and/or other obstructions. In some examples, the neural networks 992, the updated neural networks 992, and/or the map information 994 may have resulted from new training and/or experiences represented in data received from any number of vehicles in the environment, and/or based on training performed at a datacenter (e.g., using the server(s) 978 and/or other servers).

[0196] The server(s) 978 may be used to train machine learning models (e.g., neural networks) based on training data. The training data may be generated by the vehicles, and/or may be generated in a simulation (e.g., using a game engine). In some examples, the training data is tagged (e.g., where the neural network benefits from supervised learning) and/or undergoes other pre-processing, while in other examples the training data is not tagged and/or pre-processed (e.g., where the neural network does not require supervised learning). Training may be executed according to any one or more classes of machine learning techniques, including, without limitation, classes such as: supervised training, semi-supervised training, unsupervised training, self-learning, reinforcement learning, federated learning, transfer learning, feature learning (including principal component and cluster analyses), multi-linear subspace learning, manifold learning, representation learning (including sparse dictionary learning), rule-based machine learning, anomaly detection, and any variants or combinations thereof. Once the machine learning models are trained, the machine learning models may be used by the vehicles (e.g., transmitted to the vehicles over the network(s) 990, and/or the machine learning models may be used by the server(s) 978 to remotely monitor the vehicles).

[0197] In some examples, the server(s) 978 may receive data from the vehicles and apply the data to up-to-date real-time neural networks for real-time intelligent inferencing. The server(s) 978 may include deep-learning supercomputers and/or dedicated AI computers powered by GPU(s) 984, such as a DGX and DGX Station machines developed by NVIDIA. However, in some examples, the server(s) 978 may include deep learning infrastructure that use only CPU-powered datacenters.

[0198] The deep-learning infrastructure of the server(s) 978 may be capable of fast, real-time inferencing, and may use that capability to evaluate and verify the health of the processors, software, and/or associated hardware in the vehicle 900. For example, the deep-learning infrastructure may receive periodic updates from the vehicle 900, such as a sequence of images and/or objects that the vehicle 900 has located in that sequence of images (e.g., via computer vision

and/or other machine learning object classification techniques). The deep-learning infrastructure may run its own neural network to identify the objects and compare them with the objects identified by the vehicle **900** and, if the results do not match and the infrastructure concludes that the AI in the vehicle **900** is malfunctioning, the server(s) **978** may transmit a signal to the vehicle **900** instructing a fail-safe computer of the vehicle **900** to assume control, notify the passengers, and complete a safe parking maneuver.

[0199] For inferencing, the server(s) **978** may include the GPU(s) **984** and one or more programmable inference accelerators (e.g., NVIDIA's TensorRT). The combination of GPU-powered servers and inference acceleration may make real-time responsiveness possible. In other examples, such as where performance is less critical, servers powered by CPUs, FPGAs, and other processors may be used for inferencing.

Example Computing Device

[0200] FIG. **10** is a block diagram of an example computing device(s) **1000** suitable for use in implementing some embodiments of the present disclosure. Computing device **1000** may include an interconnect system **1002** that directly or indirectly couples the following devices: memory **1004**, one or more central processing units (CPUs) **1006**, one or more graphics processing units (GPUs) **1008**, a communication interface **1010**, input/output (I/O) ports **1012**, input/output components **1014**, a power supply **1016**, one or more presentation components **1018** (e.g., display(s)), and one or more logic units **1020**. In at least one embodiment, the computing device(s) **1000** may comprise one or more virtual machines (VMs), and/or any of the components thereof may comprise virtual components (e.g., virtual hardware components). For non-limiting examples, one or more of the GPUs **1008** may comprise one or more vGPUs, one or more of the CPUs **1006** may comprise one or more vCPUs, and/or one or more of the logic units **1020** may comprise one or more virtual logic units. As such, a computing device(s) **1000** may include discrete components (e.g., a full GPU dedicated to the computing device **1000**), virtual components (e.g., a portion of a GPU dedicated to the computing device **1000**), or a combination thereof.

[0201] Although the various blocks of FIG. **10** are shown as connected via the interconnect system **1002** with lines, this is not intended to be limiting and is for clarity only. For example, in some embodiments, a presentation component **1018**, such as a display device, may be considered an I/O component **1014** (e.g., if the display is a touch screen). As another example, the CPUs **1006** and/or GPUs **1008** may include memory (e.g., the memory **1004** may be representative of a storage device in addition to the memory of the GPUs **1008**, the CPUs **1006**, and/or other components). In other words, the computing device of FIG. **10** is merely illustrative. Distinction is not made between such categories as "workstation," "server," "laptop," "desktop," "tablet," "client device," "mobile device," "hand-held device," "game console," "electronic control unit (ECU)," "virtual reality system," and/or other device or system types, as all are contemplated within the scope of the computing device of FIG. **10**.

[0202] The interconnect system **1002** may represent one or more links or busses, such as an address bus, a data bus, a control bus, or a combination thereof. The interconnect

system **1002** may include one or more bus or link types, such as an industry standard architecture (ISA) bus, an extended industry standard architecture (EISA) bus, a video electronics standards association (VESA) bus, a peripheral component interconnect (PCI) bus, a peripheral component interconnect express (PCIe) bus, and/or another type of bus or link. In some embodiments, there are direct connections between components. As an example, the CPU **1006** may be directly connected to the memory **1004**. Further, the CPU **1006** may be directly connected to the GPU **1008**. Where there is direct, or point-to-point connection between components, the interconnect system **1002** may include a PCIe link to carry out the connection. In these examples, a PCI bus need not be included in the computing device **1000**.

[0203] The memory **1004** may include any of a variety of computer-readable media. The computer-readable media may be any available media that may be accessed by the computing device **1000**. The computer-readable media may include both volatile and nonvolatile media, and removable and non-removable media. By way of example, and not limitation, the computer-readable media may comprise computer-storage media and communication media.

[0204] The computer-storage media may include both volatile and nonvolatile media and/or removable and non-removable media implemented in any method or technology for storage of information such as computer-readable instructions, data structures, program modules, and/or other data types. For example, the memory **1004** may store computer-readable instructions (e.g., that represent a program(s) and/or a program element(s), such as an operating system. Computer-storage media may include, but is not limited to, RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium which may be used to store the desired information and which may be accessed by computing device **1000**. As used herein, computer storage media does not comprise signals per se.

[0205] The computer storage media may embody computer-readable instructions, data structures, program modules, and/or other data types in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. The term "modulated data signal" may refer to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal. By way of example, and not limitation, the computer storage media may include wired media such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media. Combinations of any of the above should also be included within the scope of computer-readable media.

[0206] The CPU(s) **1006** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **1000** to perform one or more of the methods and/or processes described herein. The CPU(s) **1006** may each include one or more cores (e.g., one, two, four, eight, twenty-eight, seventy-two, etc.) that are capable of handling a multitude of software threads simultaneously. The CPU(s) **1006** may include any type of processor, and may include different types of processors depending on the type of computing device **1000** implemented (e.g., processors with fewer cores

for mobile devices and processors with more cores for servers). For example, depending on the type of computing device **1000**, the processor may be an Advanced RISC Machines (ARM) processor implemented using Reduced Instruction Set Computing (RISC) or an x86 processor implemented using Complex Instruction Set Computing (CISC). The computing device **1000** may include one or more CPUs **1006** in addition to one or more microprocessors or supplementary co-processors, such as math co-processors.

[0207] In addition to or alternatively from the CPU(s) **1006**, the GPU(s) **1008** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **1000** to perform one or more of the methods and/or processes described herein. One or more of the GPU(s) **1008** may be an integrated GPU (e.g., with one or more of the CPU(s) **1006** and/or one or more of the GPU(s) **1008** may be a discrete GPU. In embodiments, one or more of the GPU(s) **1008** may be a coprocessor of one or more of the CPU(s) **1006**. The GPU(s) **1008** may be used by the computing device **1000** to render graphics (e.g., 3D graphics) or perform general purpose computations. For example, the GPU(s) **1008** may be used for General-Purpose computing on GPUs (GPGPU). The GPU(s) **1008** may include hundreds or thousands of cores that are capable of handling hundreds or thousands of software threads simultaneously. The GPU(s) **1008** may generate pixel data for output images in response to rendering commands (e.g., rendering commands from the CPU(s) **1006** received via a host interface). The GPU(s) **1008** may include graphics memory, such as display memory, for storing pixel data or any other suitable data, such as GPGPU data. The display memory may be included as part of the memory **1004**. The GPU(s) **1008** may include two or more GPUs operating in parallel (e.g., via a link). The link may directly connect the GPUs (e.g., using NVLINK) or may connect the GPUs through a switch (e.g., using NVSwitch). When combined together, each GPU **1008** may generate pixel data or GPGPU data for different portions of an output or for different outputs (e.g., a first GPU for a first image and a second GPU for a second image). Each GPU may include its own memory, or may share memory with other GPUs.

[0208] In addition to or alternatively from the CPU(s) **1006** and/or the GPU(s) **1008**, the logic unit(s) **1020** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **1000** to perform one or more of the methods and/or processes described herein. In embodiments, the CPU(s) **1006**, the GPU(s) **1008**, and/or the logic unit(s) **1020** may discretely or jointly perform any combination of the methods, processes and/or portions thereof. One or more of the logic units **1020** may be part of and/or integrated in one or more of the CPU(s) **1006** and/or the GPU(s) **1008** and/or one or more of the logic units **1020** may be discrete components or otherwise external to the CPU(s) **1006** and/or the GPU(s) **1008**. In embodiments, one or more of the logic units **1020** may be a coprocessor of one or more of the CPU(s) **1006** and/or one or more of the GPU(s) **1008**.

[0209] Examples of the logic unit(s) **1020** include one or more processing cores and/or components thereof, such as Data Processing Units (DPUs), Tensor Cores (TCs), Tensor Processing Units (TPUs), Pixel Visual Cores (PVCs), Vision Processing Units (VPUs), Graphics Processing Clusters (GPCs), Texture Processing Clusters (TPCs), Streaming

Multiprocessors (SMs), Tree Traversal Units (TTUs), Artificial Intelligence Accelerators (AIAs), Deep Learning Accelerators (DLAs), Arithmetic-Logic Units (ALUs), Application-Specific Integrated Circuits (ASICs), Floating Point Units (FPUs), input/output (I/O) elements, peripheral component interconnect (PCI) or peripheral component interconnect express (PCIe) elements, and/or the like.

[0210] The communication interface **1010** may include one or more receivers, transmitters, and/or transceivers that enable the computing device **1000** to communicate with other computing devices via an electronic communication network, included wired and/or wireless communications. The communication interface **1010** may include components and functionality to enable communication over any of a number of different networks, such as wireless networks (e.g., Wi-Fi, Z-Wave, Bluetooth, Bluetooth LE, ZigBee, etc.), wired networks (e.g., communicating over Ethernet or InfiniBand), low-power wide-area networks (e.g., LoRaWAN, SigFox, etc.), and/or the Internet. In one or more embodiments, logic unit(s) **1020** and/or communication interface **1010** may include one or more data processing units (DPUs) to transmit data received over a network and/or through interconnect system **1002** directly to (e.g., a memory of) one or more GPU(s) **1008**.

[0211] The I/O ports **1012** may enable the computing device **1000** to be logically coupled to other devices including the I/O components **1014**, the presentation component(s) **1018**, and/or other components, some of which may be built in to (e.g., integrated in) the computing device **1000**. Illustrative I/O components **1014** include a microphone, mouse, keyboard, joystick, game pad, game controller, satellite dish, scanner, printer, wireless device, etc. The I/O components **1014** may provide a natural user interface (NUI) that processes air gestures, voice, or other physiological inputs generated by a user. In some instances, inputs may be transmitted to an appropriate network element for further processing. An NUI may implement any combination of speech recognition, stylus recognition, facial recognition, biometric recognition, gesture recognition both on screen and adjacent to the screen, air gestures, head and eye tracking, and touch recognition (as described in more detail below) associated with a display of the computing device **1000**. The computing device **1000** may include depth cameras, such as stereoscopic camera systems, infrared camera systems, RGB camera systems, touchscreen technology, and combinations of these, for gesture detection and recognition. Additionally, the computing device **1000** may include accelerometers or gyroscopes (e.g., as part of an inertia measurement unit (IMU)) that enable detection of motion. In some examples, the output of the accelerometers or gyroscopes may be used by the computing device **1000** to render immersive augmented reality or virtual reality.

[0212] The power supply **1016** may include a hard-wired power supply, a battery power supply, or a combination thereof. The power supply **1016** may provide power to the computing device **1000** to enable the components of the computing device **1000** to operate.

[0213] The presentation component(s) **1018** may include a display (e.g., a monitor, a touch screen, a television screen, a heads-up-display (HUD), other display types, or a combination thereof), speakers, and/or other presentation components. The presentation component(s) **1018** may receive

data from other components (e.g., the GPU(s) 1008, the CPU(s) 1006, DPUs, etc.), and output the data (e.g., as an image, video, sound, etc.).

Example Data Center

[0214] FIG. 11 illustrates an example data center 1100 that may be used in at least one embodiment of the present disclosure. The data center 1100 may include a data center infrastructure layer 1110, a framework layer 1120, a software layer 1130, and/or an application layer 1140.

[0215] As shown in FIG. 11, the data center infrastructure layer 1110 may include a resource orchestrator 1112, grouped computing resources 1114, and node computing resources (“node C.R.s”) 1116(1)-1116(N), where “N” represents any whole, positive integer. In at least one embodiment, node C.R.s 1116(1)-1116(N) may include, but are not limited to, any number of central processing units (CPUs) or other processors (including DPUs, accelerators, field programmable gate arrays (FPGAs), graphics processors or graphics processing units (GPUs), etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output (NW I/O) devices, network switches, virtual machines (VMs), power modules, and/or cooling modules, etc. In some embodiments, one or more node C.R.s from among node C.R.s 1116(1)-1116(N) may correspond to a server having one or more of the above-mentioned computing resources. In addition, in some embodiments, the node C.R.s 1116(1)-1116(N) may include one or more virtual components, such as vGPUs, vCPUs, and/or the like, and/or one or more of the node C.R.s 1116(1)-1116(N) may correspond to a virtual machine (VM).

[0216] In at least one embodiment, grouped computing resources 1114 may include separate groupings of node C.R.s 1116 housed within one or more racks (not shown), or many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s 1116 within grouped computing resources 1114 may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s 1116 including CPUs, GPUs, DPUs, and/or other processors may be grouped within one or more racks to provide compute resources to support one or more workloads. The one or more racks may also include any number of power modules, cooling modules, and/or network switches, in any combination.

[0217] The resource orchestrator 1112 may configure or otherwise control one or more node C.R.s 1116(1)-1116(N) and/or grouped computing resources 1114. In at least one embodiment, resource orchestrator 1112 may include a software design infrastructure (SDI) management entity for the data center 1100. The resource orchestrator 1112 may include hardware, software, or some combination thereof.

[0218] In at least one embodiment, as shown in FIG. 11, framework layer 1120 may include a job scheduler 1132, a configuration manager 1134, a resource manager 1136, and/or a distributed file system 1138. The framework layer 1120 may include a framework to support software 1132 of software layer 1130 and/or one or more application(s) 1142 of application layer 1140. The software 1132 or application(s) 1142 may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. The

framework layer 1120 may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter “Spark”) that may utilize distributed file system 1138 for large-scale data processing (e.g., “big data”). In at least one embodiment, job scheduler 1132 may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center 1100. The configuration manager 1134 may be capable of configuring different layers such as software layer 1130 and framework layer 1120 including Spark and distributed file system 1138 for supporting large-scale data processing. The resource manager 1136 may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system 1138 and job scheduler 1132. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource 1114 at data center infrastructure layer 1110. The resource manager 1136 may coordinate with resource orchestrator 1112 to manage these mapped or allocated computing resources.

[0219] In at least one embodiment, software 1132 included in software layer 1130 may include software used by at least portions of node C.R.s 1116(1)-1116(N), grouped computing resources 1114, and/or distributed file system 1138 of framework layer 1120. One or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0220] In at least one embodiment, application(s) 1142 included in application layer 1140 may include one or more types of applications used by at least portions of node C.R.s 1116(1)-1116(N), grouped computing resources 1114, and/or distributed file system 1138 of framework layer 1120. One or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.), and/or other machine learning applications used in conjunction with one or more embodiments.

[0221] In at least one embodiment, any of configuration manager 1134, resource manager 1136, and resource orchestrator 1112 may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. Self-modifying actions may relieve a data center operator of data center 1100 from making possibly bad configuration decisions and possibly avoiding underutilized and/or poor performing portions of a data center.

[0222] The data center 1100 may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein. For example, a machine learning model(s) may be trained by calculating weight parameters according to a neural network architecture using software and/or computing resources described above with respect to the data center 1100. In at least one embodiment, trained or deployed machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to the data center 1100 by using weight parameters calculated through one or more training techniques, such as but not limited to those described herein.

[0223] In at least one embodiment, the data center **1100** may use CPUs, application-specific integrated circuits (ASICs), GPUs, FPGAs, and/or other hardware (or virtual compute resources corresponding thereto) to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

Example Network Environments

[0224] Network environments suitable for use in implementing embodiments of the disclosure may include one or more client devices, servers, network attached storage (NAS), other backend devices, and/or other device types. The client devices, servers, and/or other device types (e.g., each device) may be implemented on one or more instances of the computing device(s) **1000** of FIG. **10**—e.g., each device may include similar components, features, and/or functionality of the computing device(s) **1000**. In addition, where backend devices (e.g., servers, NAS, etc.) are implemented, the backend devices may be included as part of a data center **1100**, an example of which is described in more detail herein with respect to FIG. **11**.

[0225] Components of a network environment may communicate with each other via a network(s), which may be wired, wireless, or both. The network may include multiple networks, or a network of networks. By way of example, the network may include one or more Wide Area Networks (WANs), one or more Local Area Networks (LANs), one or more public networks such as the Internet and/or a public switched telephone network (PSTN), and/or one or more private networks. Where the network includes a wireless telecommunications network, components such as a base station, a communications tower, or even access points (as well as other components) may provide wireless connectivity.

[0226] Compatible network environments may include one or more peer-to-peer network environments—in which case a server may not be included in a network environment—and one or more client-server network environments—in which case one or more servers may be included in a network environment. In peer-to-peer network environments, functionality described herein with respect to a server(s) may be implemented on any number of client devices.

[0227] In at least one embodiment, a network environment may include one or more cloud-based network environments, a distributed computing environment, a combination thereof, etc. A cloud-based network environment may include a framework layer, a job scheduler, a resource manager, and a distributed file system implemented on one or more of servers, which may include one or more core network servers and/or edge servers. A framework layer may include a framework to support software of a software layer and/or one or more application(s) of an application layer. The software or application(s) may respectively include web-based service software or applications. In embodiments, one or more of the client devices may use the web-based service software or applications (e.g., by accessing the service software and/or applications via one or more application programming interfaces (APIs)). The framework layer may be, but is not limited to, a type of free and

open-source software web application framework such as that may use a distributed file system for large-scale data processing (e.g., “big data”).

[0228] A cloud-based network environment may provide cloud computing and/or cloud storage that carries out any combination of computing and/or data storage functions described herein (or one or more portions thereof). Any of these various functions may be distributed over multiple locations from central or core servers (e.g., of one or more data centers that may be distributed across a state, a region, a country, the globe, etc.). If a connection to a user (e.g., a client device) is relatively close to an edge server(s), a core server(s) may designate at least a portion of the functionality to the edge server(s). A cloud-based network environment may be private (e.g., limited to a single organization), may be public (e.g., available to many organizations), and/or a combination thereof (e.g., a hybrid cloud environment).

[0229] The client device(s) may include at least some of the components, features, and functionality of the example computing device(s) **1000** described herein with respect to FIG. **10**. By way of example and not limitation, a client device may be embodied as a Personal Computer (PC), a laptop computer, a mobile device, a smartphone, a tablet computer, a smart watch, a wearable computer, a Personal Digital Assistant (PDA), an MP3 player, a virtual reality headset, a Global Positioning System (GPS) or device, a video player, a video camera, a surveillance device or system, a vehicle, a boat, a flying vessel, a virtual machine, a drone, a robot, a handheld communications device, a hospital device, a gaming device or system, an entertainment system, a vehicle computer system, an embedded system controller, a remote control, an appliance, a consumer electronic device, a workstation, an edge device, any combination of these delineated devices, or any other suitable device.

[0230] The disclosure may be described in the general context of computer code or machine-useable instructions, including computer-executable instructions such as program modules, being executed by a computer or other machine, such as a personal data assistant or other handheld device. Generally, program modules including routines, programs, objects, components, data structures, etc., refer to code that perform particular tasks or implement particular abstract data types. The disclosure may be practiced in a variety of system configurations, including hand-held devices, consumer electronics, general-purpose computers, more specialty computing devices, etc. The disclosure may also be practiced in distributed computing environments where tasks are performed by remote-processing devices that are linked through a communications network.

[0231] As used herein, a recitation of “and/or” with respect to two or more elements should be interpreted to mean only one element, or a combination of elements. For example, “element A, element B, and/or element C” may include only element A, only element B, only element C, element A and element B, element A and element C, element B and element C, or elements A, B, and C. In addition, “at least one of element A or element B” may include at least one of element A, at least one of element B, or at least one of element A and at least one of element B. Further, “at least one of element A and element B” may include at least one of element A, at least one of element B, or at least one of element A and at least one of element B.

[0232] The subject matter of the present disclosure is described with specificity herein to meet statutory require-

ments. However, the description itself is not intended to limit the scope of this disclosure. Rather, the inventors have contemplated that the claimed subject matter might also be embodied in other ways, to include different steps or combinations of steps similar to the ones described in this document, in conjunction with other present or future technologies. Moreover, although the terms “step” and/or “block” may be used herein to connote different elements of methods employed, the terms should not be interpreted as implying any particular order among or between various steps herein disclosed unless and except when the order of individual steps is explicitly described.

What is claimed is:

1. An autonomous or semi-autonomous machine comprising:

one or more central processing units (CPUs);
one or more graphics processing units (GPUs);
one or more hardware accelerators; and
one or more sensors having one or more fields of view or one or more sensory fields,

wherein the autonomous or semi-autonomous machine is to perform one or more planning, control, or navigation operations based at least on one or more associations between one or more objects and one or more lanes, wherein the one or more associations are determined based at least on a neural network processing first data associated with the one or more objects, second data associated with the one or more lanes, and sensor data obtained using the one or more sensors.

2. The autonomous or semi-autonomous machine of claim 1, wherein:

the first data represents one or more first locations associated with the one or more objects; and
the second data represents one or more second locations associated with the one or more lanes.

3. The autonomous or semi-autonomous machine of claim 1, wherein:

the first data represents one or more bounding shapes associated with the one or more objects; and
the second data represents one or more lane graphs associated with the one or more lanes.

4. The autonomous or semi-autonomous machine of claim 1, wherein:

the first data represents one or more first images representing the one or more objects; and
the second data represents one or more second images representing the one or more lanes.

5. The autonomous or semi-autonomous machine of claim 1, wherein:

the sensor data represents the one or more objects and the one or more lanes;
the first data represents the one or more objects without representing the one or more lanes; and
the second data represents the one or more lanes without representing the one or more objects.

6. The autonomous or semi-autonomous machine of claim 1, wherein the autonomous or semi-autonomous machine is further to generate at least one of the first data or the second data based at least on processing the sensor data.

7. The autonomous or semi-autonomous machine of claim 1, wherein the sensor data represents the one or more objects and the one or more lanes.

8. The autonomous or semi-autonomous machine of claim 1, wherein:

the second data represents one or more classifications relating the one or more lanes with respect to a lane associated with the autonomous or semi-autonomous machine; and

the autonomous or semi-autonomous machine is further to perform the one or more planning, control, and navigation operations based at least on the one or more classifications.

9. A system comprising:

one or more processors to:

obtain first data representative of one or more first locations associated with one or more objects within an environment and second data representative of one or more second locations associated with one or more lanes within the environment;

determine, based at least on a neural network processing the first data, the second data, and sensor data representative of the environment, one or more associations between the one or more objects and the one or more lanes; and

perform, based at least on the one or more associations, one or more planning, control, or navigation operations associated with a machine.

10. The system of claim 9, wherein the one or more processors are further to input the first data and the second data along with the sensor data into the neural network for processing.

11. The system of claim 9, wherein:

the first data represents one or more bounding shapes indicating the one or more first locations associated with the one or more objects; and

the second data represents one or more lane graphs indicating the one or more second locations associated with the one or more lanes.

12. The system of claim 9, wherein:

the first data represents one or more first images indicating the one or more first locations associated with the one or more objects; and

the second data represents one or more second images indicating the one or more second locations associated with the one or more lanes.

13. The system of claim 9, wherein:

the sensor data represents the one or more first locations associated with the one or more objects and the one or more second locations associated with the one or more lanes;

the first data does not represent the one or more second locations associated with the one or more lanes; and

the second data does not represent the one or more first locations associated with the one or more objects.

14. The system of claim 9, wherein the one or more processors are further to generate at least one of the first data or the second data based at least on processing the sensor data.

15. The system of claim 9, wherein:

the second data further represents one or more classifications relating the one or more lanes with respect to a lane associated with the machine; and

the one or more planning, control, and navigation operations are further performed based at least on the one or more classifications.

16. The system of claim 9, wherein the system is comprised in at least one of:

a control system for an autonomous or semi-autonomous machine;
a perception system for an autonomous or semi-autonomous machine;
a system for performing simulation operations;
a system for performing deep learning operations;
a system implemented using a collaborative content creation platform for 3D assets;
a system implemented using an edge device;
a system implemented using a robot;
a system incorporating one or more virtual machines (VMs);
a system implemented at least partially in a data center; or
a system implemented at least partially using cloud computing resources.

17. A method comprising:

applying, to a neural network, sensor data obtained using one or more sensors, first data representative of first information associated with one or more objects, and second data representative of second information associated with one or more lanes;

determining, based at least on the neural network processing the sensor data, the first data, and the second data, one or more associations between the one or more objects and the one or more lanes; and

performing, based at least on the one or more associations, one or more planning, control, or navigation operations associated with a machine.

18. The method of claim 17, wherein:

the first information indicates one or more first locations associated with the one or more objects; and
the second information indicates one or more second locations associated with the one or more lanes.

19. The method of claim 17, wherein:

the first information indicates one or more bounding shapes associated with the one or more objects; and
the second information indicates one or more lane graphs associated with the one or more lanes.

20. The method of claim 17, further comprising processing the sensor data to generate at least one of the first data or the second data.

* * * * *